

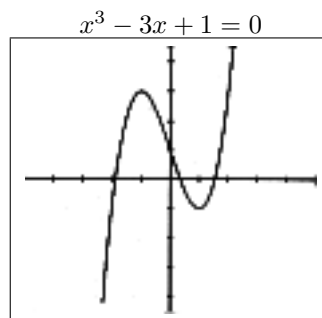
Module 1

Roots of Single Equations

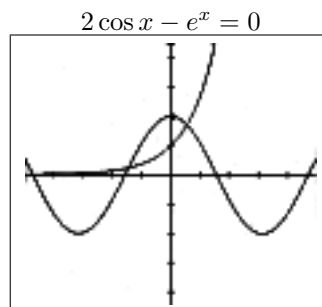
Problem: Find roots of an equation of the form

$$f(x) = 0,$$

where $f(x)$ is a piecewise continuous function of x having numerical coefficients. The function $f(x)$ may be *algebraic* (i.e., a polynomial in x), as in



or it may be *transcendental*, as in



We will assume in general that $f(x)$ is differentiable and that the labor of obtaining the numerical value of $f(x)$ for any value of x is not excessive.

Example: Longest ladder

1.1 Polynomial Equations

1.1.1 General Form

$$\begin{aligned}
 f(x) &= x^n + a_1x^{n-1} + a_2x^{n-2} + \cdots + a_{n-1}x + a_n = 0 \\
 &= \sum_{i=0}^n a_ix^{n-i} = 0, \quad a_0 = 1, \quad a_n \neq 0 \\
 &= (x - x_1)(x - x_2)(x - x_3) \cdots (x - x_n) \\
 &= \prod_{i=1}^n (x - x_i), \quad x_i \neq 0
 \end{aligned}$$

1.1.2 Some Properties

1. There are n roots.
2. The roots may be real or complex; complex roots occur in conjugate pairs. There is at least one real root if n is odd.
3. Equal roots may exist. If the root x_j occurs m times, it is said to be of *multiplicity* m .
4. The number of positive roots is equal to the number of sign changes of the coefficients of $f(x)$ or is less than this by an even number. The number of negative roots is equal to the number of sign changes of the coefficients of $f(-x)$ or is less than this by an even number.

Graphical example: cubic equation with (a) 3 distinct real roots (b) 1 distinct and 2 equal real roots (c) 1 real and a pair of complex roots (d) 3 equal real roots

1.1.3 History

A little bit of history on the quest for solutions to polynomial equations:

- The quadratic formula for solving quadratic equations has been known since ancient times. It is easily derived by completing the square.
- In the 1500s, earnest search (motivated by prize money) for a similar general procedure to solve cubic and quartic equations was started in Italian universities: Tartaglia, Cardano, Ferrari.
- For the next three centuries, mathematicians used every ingenious way to find general solutions to quintic and higher degree equations: all to no avail.
- In 1824, Niels Henrik Abel, a Norwegian mathematician, proved that it is impossible to solve quintic equations in terms of a universal, finite procedure involving only elementary operations of arithmetic and algebra. He was 22 years old.
- In 1832, Evariste Galois, a French mathematician, the night before he was killed in a duel, wrote down the proof that there is no general, finite procedure to solve all equations of degree five or higher in terms of elementary operations. He was 20.

1.2 Transcendental Equations

A transcendental equation may have a finite or an infinite number of roots. The roots may be complex or real.

Examples:

$$\begin{aligned}\sin x - 2 &= 0 \\ \sin x - \frac{1}{2} &= 0 \\ \sin x - \frac{x}{2} &= 0\end{aligned}$$

There are no formulas, in the manner of the quadratic formula, for solving transcendental equations. In general, we solve these equations using numerical methods.

1.3 Iterative Numerical Methods

Definition: An iterative numerical method for finding a root, say $\bar{x}$, of the equation $f(x) = 0$ is a numerical method in which a sequence of estimates of $\bar{x}$ is generated such that

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \cdots \rightarrow x_k \rightarrow x_{k+1} \rightarrow \cdots$$

If the sequence of estimates leads to $\bar{x}$, the method is said to *converge*; otherwise, it is said to *diverge*. Using an iterative numerical method involves essentially two tasks, viz.,

- a. finding an initial estimate, x_0 , for the desired root
- b. refining the estimate to some prescribed degree of accuracy

1.4 Finding an Initial Estimate of the Root

1. **physical consideration**—the equation to be solved may be descriptive of a physical phenomenon from which we can deduce an approximate value of the solution

Example: the Beattie-Bridgeman equation of state for an imperfect (actual) gas is

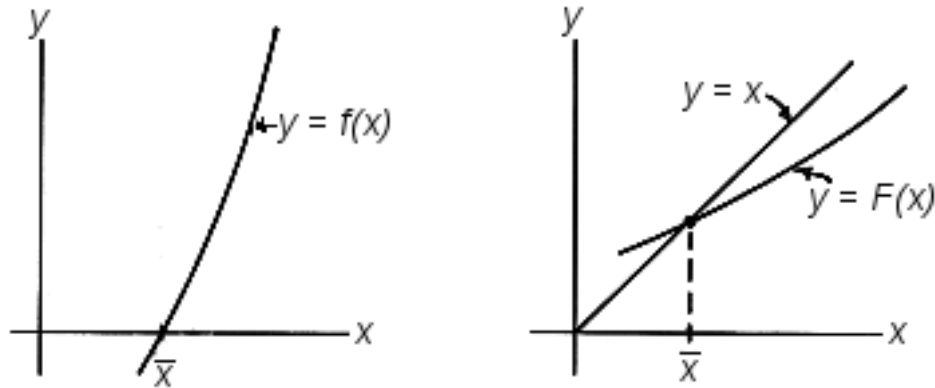
$$PV = RT + \frac{\beta}{V} + \frac{\gamma}{V^2} + \frac{\delta}{V^3},$$

where β , γ and δ are empirical constants for the gas. To obtain an initial estimate of V given P and T , we could assume that the gas is an ideal gas. Then $PV = RT$ and $V_0 = \frac{RT}{P}$.

2. **graphical procedure**—plot the given equation, as a whole or by parts
3. **another numerical method**—e.g., Graeffe's Root-Squaring Method, for polynomial equations.
4. others

1.5 Method of Successive Substitutions

To apply the MoSS, we first transform the given equation $f(x) = 0$ into the form $x = F(x)$. The effect of this transformation is graphically shown below:

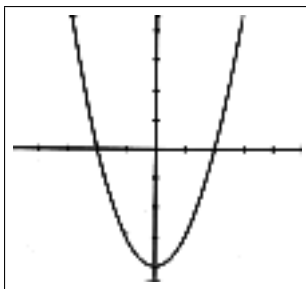


In principle, there are many possible ways of generating the form $x = F(x)$ for a given $f(x) = 0$. As a rule, we should choose $F(x)$ to be as simple as possible. Depending on the form of $F(x)$ that we choose, MoSS may:

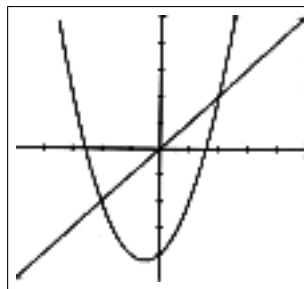
- converge to the desired root
- converge, but not to the desired root
- diverge

Example:

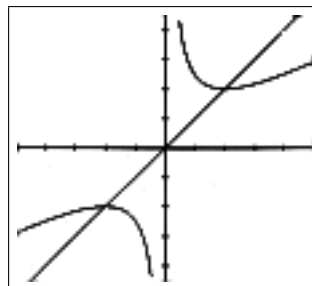
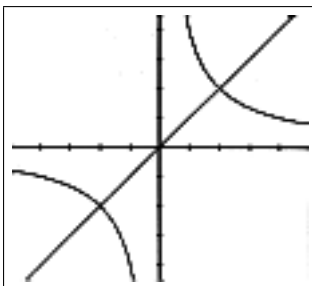
$$f(x) = x^2 - 4 = 0$$



$$x = x^2 + x - 4 \quad \text{Add } x \text{ to both sides}$$



$$x = \frac{4}{x} \quad \text{Add 4 to both sides and divide by } x \quad x = \frac{1}{2} \left(x + \frac{4}{x} \right) \quad \text{Add } x^2 + 4 \text{ to both sides and divide by } 2x$$



1.5.1 Summary

Form of the equation: $x = F(x)$

Iterative formula: $x_{k+1} = F(x_k)$

Sufficient condition for convergence: $|F'(x)| < 1$

Criteria for terminating the iterations:

- a. Absolute error criterion: $|x_{k+1} - x_k| < \varepsilon$ or
- b. Relative error criterion: $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| < \varepsilon$ or
- c. others and
- d. A prescribed maximum number of iterations, $itmax$

1.5.2 EASY Implementation

procedure MOSS($\varepsilon, itmax, xold$)

$iter \leftarrow 0$

 output $iter, xold$, 'Initial estimate'

 for $iter \leftarrow 1$ to $itmax$ do

$xnew \leftarrow F(xold)$

 //function call to evaluate $F(x_k)$ //

$xerr \leftarrow |(xnew - xold)/xnew|$

 output $iter, xnew, xerr$

 if $xerr < \varepsilon$ then return

 //method converges within $itmax$ iterations//

 else $xold \leftarrow xnew$

 end for

 return

 //method fails to converge in $itmax$ iterations//

end procedure

In general, a procedure to implement an iterative numerical method, such as procedure MOSS given above, has four parts:

- a. initialization
- b. computation
- c. testing
- d. modification

Example 1. Find the roots of the equation $x^3 - 3x + 1 = 0$. Iterate until the relative error is less than $\varepsilon = 10^{-6}$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

- a. Cast the given equation into a form suitable for the MoSS, i.e., $x = F(x)$. Let

$$x = \frac{x^3 + 1}{3}$$

- b. Find initial estimates: see graph of $f(x) = x^3 - 3x + 1 = 0$

- c. Perform analysis for convergence of the chosen $F(x)$

Since $F'(x) = x^2$, then $|F'(x)| < 1$ if $|x| < 1$, or if $-1 < x < 1$. Hence, $F(x) = \frac{x^3 + 1}{3}$ will yield the middle root, but probably not the other roots.

d. EASY procedures

```
procedure MAIN()
```

```
1: input  $x_0$ 
```

```
call MOSS( $10^{-6}$ , 20,  $x_0$ )
```

```
go to 1
```

```
end procedure
```

```
procedure F( $x$ )
```

```
return  $(x^3 + 1)/3$ 
```

```
end procedure
```

```
//initial estimate//
```

e. Results:

Original equation: $x^3 - 3x + 1 = 0$

Transformed equation: $x = \frac{x^3 + 1}{3}$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of the MoSS:

k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $	k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $	k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $
0	.5000000		0	1.5000000		0	-1.8000000	
1	.3750000	.3333333	1	1.4583333	.0285714	1	-1.6106667	.1175497
2	.3509115	.0686456	2	1.3671634	.0666855	2	-1.0594891	.5202296
3	.3477369	.0091291	3	1.1851380	.1535901	3	-.0630982	5.7911022
4	.3473496	.0011152	4	.8881960	.3343204	4	.3332496	1.1893423
5	.3473028	.0001347	5	.5668969	.5667680	5	.3456697	.0359306
6	.3472971	.0000163	6	.3940616	.4385997	6	.3471011	.0041238
7	.3472964	.0000020	7	.3537306	.1140163	7	.3472728	.0004945
8	.3472964	.0000002	8	.3480869	.0162134	8	.3472935	.0000596
			9	.3473919	.0020005	9	.3472960	.0000072
			10	.3473079	.0002420	10	.3472963	.0000009
			11	.3472977	.0000292			
			12	.3472965	.0000035			
			13	.3472964	.0000004			

Original equation: $x^3 - 3x + 1 = 0$

Transformed equation: $x = \sqrt[3]{3x - 1}$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of the MoSS:

k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $	k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $
0	1.5000000		0	.5000000	
1	1.5182945	.0120494	1	.7937005	.3700395
2	1.5261895	.0051730	2	1.1136324	.2872868
3	1.5295715	.0022111	3	1.3277841	.1612850
4	1.5310157	.0009433	4	1.4395768	.0776567
5	1.5316315	.0004021	5	1.4916170	.0348884
6	1.5318940	.0001713	6	1.5146492	.0152063
7	1.5320059	.0000730	7	1.5246229	.0065417
8	1.5320535	.0000311	8	1.5289016	.0027985
9	1.5320738	.0000133	9	1.5307298	.0011944
10	1.5320825	.0000056	10	1.5315097	.0005092
11	1.5320862	.0000024	11	1.5318421	.0002170
12	1.5320877	.0000010	12	1.5319837	.0000925
13	1.5320884	.0000004	13	1.5320441	.0000394
			14	1.5320698	.0000168
			15	1.5320808	.0000071
			16	1.5320854	.0000030
			17	1.5320874	.0000013
			18	1.5320883	.0000006

Original equation: $x^3 - 3x + 1 = 0$

Transformed equation: $x = -\sqrt[3]{3x - 1}$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of the MoSS:

k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $
0	-1.8000000	
1	-1.8566355	.0305044
2	-1.8729222	.0086958
3	-1.8775536	.0024668
4	-1.8788665	.0006988
5	-1.8792384	.0001979
6	-1.8793437	.0000560
7	-1.8793735	.0000159
8	-1.8793819	.0000045
9	-1.8793843	.0000013
10	-1.8793850	.0000004

Original equation: $x^3 - 3x + 1 = 0$

Transformed equation: $x = \frac{2x^3 - 1}{3x^2 - 3}$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of the MoSS:

k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $	k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $	k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $
0	.5000000		0	-1.8000000		0	1.5000000	
1	.3333333	.5000000	1	-1.8845238	.0448515	1	1.5333333	.0217391
2	.3472222	.0400000	2	-1.8794047	.0027238	2	1.5320906	.0008111
3	.3472964	.0002135	3	-1.8793852	.0000104	3	1.5320889	.0000011
4	.3472964	.0000000	4	-1.8793852	.0000000	4	1.5320889	.0000000

Example 2. For turbulent flow ($Re > 2000$) of a fluid in a smooth pipe, the relationship between the friction factor f and the Reynold's number Re is given by the equation

$$\sqrt{\frac{1}{f}} = 1.74 \ln(Re\sqrt{f}) - 0.40$$

Calculate f for $Re = 5000, 10000, 50000, 100000, 500000, 1000000$. Iterate until the relative error is less than $\varepsilon = 10^{-6}$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

- a. Cast given equation in a form suitable for the MoSS, i.e., $f = F(f)$,

$$f = \frac{1}{[1.74 \ln(Re\sqrt{f}) - 0.40]^2}$$

- b. Given Re , find an initial estimate. To this end, we use another empirical formula relating f and Re explicitly,

$$f = \frac{0.316}{Re^{0.25}} \quad (\text{Blassius's equation})$$

Blassius's equation gives an approximate value of f for a very smooth pipe for $3000 < Re < 100000$.

c. EASY procedures

procedure MAIN()

1: input Re

// Re is assumed to be global //

$f0 \leftarrow 0.316/Re^{0.25}$

call MOSS(10^{-6} , 20, $f0$)

go to 1

end procedure

procedure F(f)

return $1.0/(1.74 * \ln(Re * \sqrt{f}) - 0.40)^2$

end procedure

d. Results:

Original equation: $\sqrt{\frac{1}{f}} = 1.74 \ln(Re \sqrt{f}) - 0.40$

Transformed equation: $f = \frac{1}{(1.74 \ln(Re \sqrt{f}) - 0.40)^2}$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of the MoSS:

k	f_k	$\left \frac{f_k - f_{k-1}}{f_k} \right $	k	f_k	$\left \frac{f_k - f_{k-1}}{f_k} \right $
0	.0375789	($Re = 5000$)	0	.0316000	($Re = 10000$)
1	.0074765	4.0263074	1	.0062784	4.0331477
2	.0096868	.2281781	2	.0079513	.2103952
3	.0092710	.0448467	3	.0076677	.0369872
4	.0093395	.0073364	4	.0077103	.0055261
5	.0093280	.0012386	5	.0077038	.0008468
6	.0093299	.0002080	6	.0077048	.0001293
7	.0093296	.0000350	7	.0077046	.0000197
8	.0093296	.0000059	8	.0077046	.0000030
9	.0093296	.0000010	9	.0077046	.0000005

e. Summary of results for the friction problem using MoSS

Reynolds number	Initial estimate	Final estimate	Number of iterations	Relative error
5000	.0375789	.0093296	9	.0000010
10000	.0316000	.0077046	9	.0000005
50000	.0211322	.0052110	8	.0000007
100000	.0177700	.0044870	8	.0000004
500000	.0118835	.0032814	8	.0000001
1000000	.0099928	.0029040	7	.0000009

1.6 Wegstein's Method

Consider again the iterative formula for the MoSS:

$$x_{k+1} = F(x_k) \quad (1.1)$$

Adding and subtracting x_k to, and from the right side of (1.1), we obtain

$$x_{k+1} = F(x_k) - x_k + x_k = \delta_k + x_k, \quad (1.2)$$

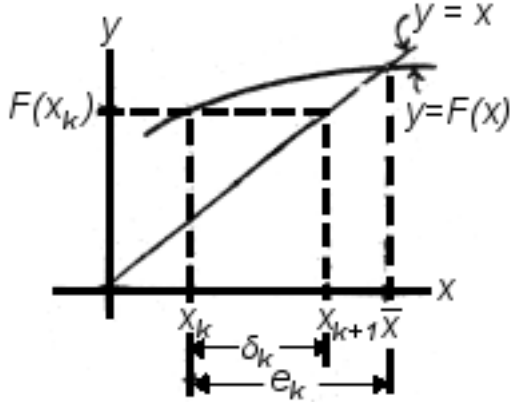
where

$$\delta_k = F(x_k) - x_k = x_{k+1} - x_k$$

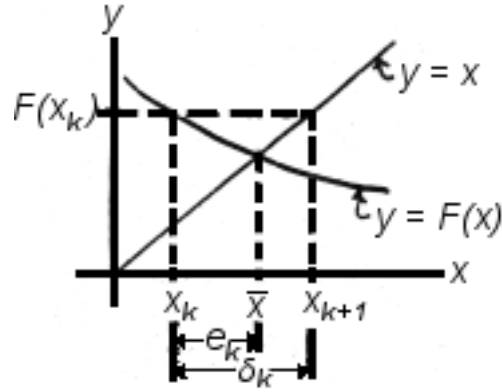
is the k th iteration estimate of the error e_k ,

$$\bar{x} - x_k$$

of the k th estimate x_k , of the root $\bar{x}$. The figures below show the geometrical representation of the quantities.



$$0 < F'(x) < 1$$



$$-1 < F'(x) < 0$$

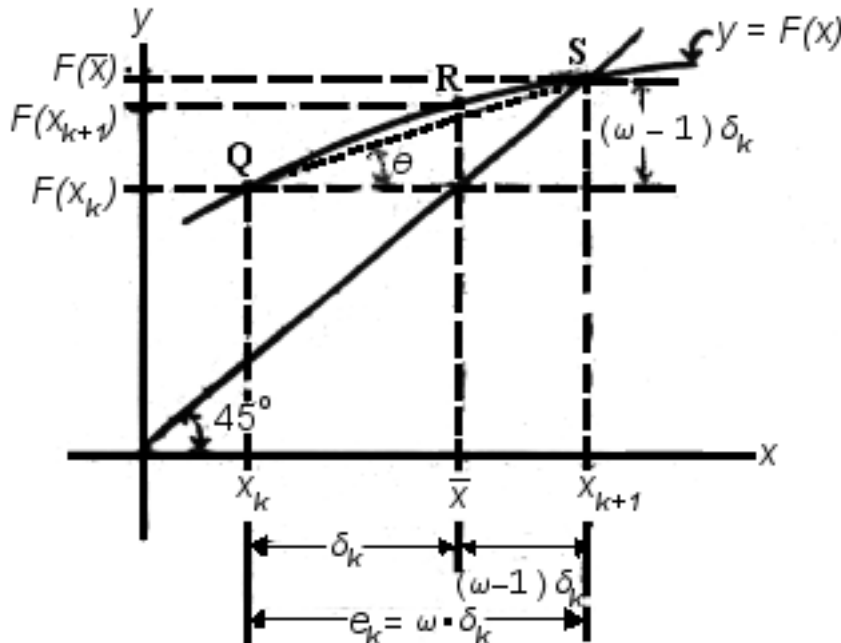
From these figures, we see that our estimate, δ_k , of the error, e_k , is either too small or too large. We either undershoot or overshoot the root. This suggests that we might be able to improve on (1.2) by applying an appropriate factor on δ_k , such that

$$x_{k+1} = x_k + \omega \cdot \delta_k \quad (1.3)$$

In (1.3), the factor ω is called a relaxation factor, and is either less than or greater than unity depending on whether δ_k over-estimates or under-estimates e_k . From the figures above, we see that the best choice for ω is $\frac{e_k}{\delta_k}$, so that in one iteration we find $\bar{x}$:

$$x_{k+1} = x_k + \frac{e_k}{\delta_k} \cdot \delta_k = x_k + e_k = \bar{x}$$

Unfortunately, this expression for ω cannot be used since e_k is not known in the first place. To arrive at a workable expression for ω , consider the figure below:



From the figure above, we obtain the following relations:

$$\tan \theta = \frac{(\omega - 1)\delta_k}{\omega \cdot \delta_k} = \frac{\omega - 1}{\omega} = \frac{F(\bar{x}) - F(x_k)}{\bar{x} - x_k} = F'(\xi),$$

where $x_k \leq \xi \leq \bar{x}$ (mean value theorem). Hence,

$$\frac{\omega - 1}{\omega} = F'(\xi)$$

or

$$\omega = \frac{1}{1 - F'(\xi)} \quad (1.4)$$

The value of ξ is unknown, but we can approximate the value of $F'(\xi)$ by the slope of the chord QR , i.e.,

$$F'(\xi) = \frac{F(x_{k+1}) - F(x_k)}{x_{k+1} - x_k} = \frac{F[F(x_k)] - F(x_k)}{F(x_k) - x_k}$$

Hence,

$$\omega = \frac{1}{1 - \frac{F[F(x_k)] - F(x_k)}{F(x_k) - x_k}} = \frac{F(x_k) - x_k}{2F(x_k) - x_k - F[F(x_k)]} \quad (1.5)$$

Substituting (1.5) in (1.3), we obtain the iterative formula for the Wegstein method:

$$x_{k+1} = x_k + \frac{F(x_k) - x_k}{2F(x_k) - x_k - F[F(x_k)]} \cdot [F(x_k) - x_k]$$

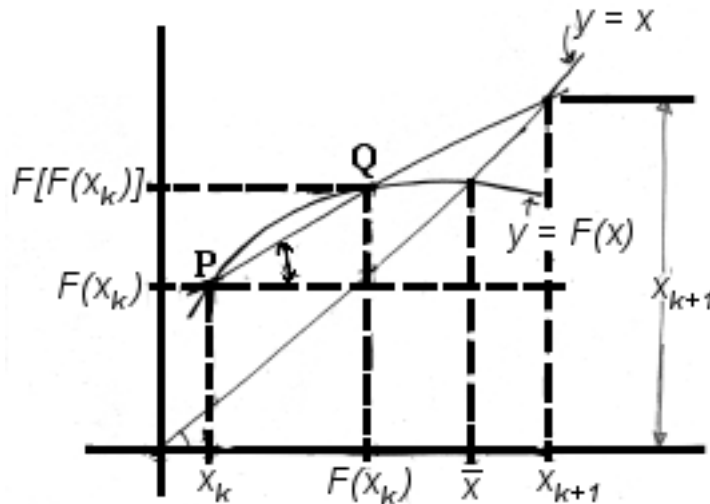
Simplifying,

$$x_{k+1} = \frac{[F(x_k)]^2 - x_k \cdot F[F(x_k)]}{2F(x_k) - x_k - F[F(x_k)]} \quad (1.6)$$

Equation (1.6) may be applied to both the convergent and divergent cases of the MoSS. From Eq. (1.4), the bounds on ω can be determined for each of these cases:

Case 1	$0 < F'(x) < 1$	$1 < \omega < \infty$	
Case 2	$-1 < F'(x) < 0$	$\frac{1}{2} < \omega < 1$	
Case 3	$F'(x) > 1$	$\omega < 0$	Positive slope, too steep
Case 4	$F'(x) < -1$	$0 < \omega < \frac{1}{2}$	Negative slope, too steep

Given below is the graphical representation of Wegstein's formula:



$$\begin{aligned}\tan \theta &= \frac{F(x_{k+1}) - F(x_k)}{x_{k+1} - x_k} = \frac{F[F(x_k)] - F(x_k)}{F(x_k) - x_k} \\ x_{k+1}[F(x_k) - x_k] - [F(x_k)]^2 + x_k \cdot F(x_k) &= x_{k+1}[F[F(x_k)] - F(x_k)] - x_k \cdot F[F(x_k)] + x_k \cdot F(x_k) \\ x_{k+1}[2F(x_k) - x_k - F[F(x_k)]] &= [F(x_k)]^2 - x_k \cdot F[F(x_k)]\end{aligned}$$

Finally:

$$x_{k+1} = \frac{[F(x_k)]^2 - x_k \cdot F[F(x_k)]}{2F(x_k) - x_k - F[F(x_k)]}$$

1.6.1 Summary

Form of the equation: $x = F(x)$

Iterative formula: $x_{k+1} = \frac{[F(x_k)]^2 - x_k \cdot F[F(x_k)]}{2F(x_k) - x_k - F[F(x_k)]}$

Convergence: may converge for cases where MoSS diverges

Criteria for termianting the iterations:

- a. Absolute error criterion: $|x_{k+1} - x_k| < \varepsilon$ or
- b. Relative error criterion: $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| < \varepsilon$ or
- c. others and
- d. A prescribed maximum number of iterations, *itmax*

1.6.2 EASY Implementation

procedure WEGSTEIN($\varepsilon, itmax, xold$)

iter $\leftarrow$ 0

output *iter*, *xold*, 'Initial estimate'

for *iter* $\leftarrow$ 1 to *itmax* do

Fxold $\leftarrow$ *F*(*xold*)

//function call to evaluate *F*(*x_k*)//

FFxold $\leftarrow$ *F*(*Fxold*)

//function call to evaluate *F*[*F*(*x_k*)]//

xnew $\leftarrow$ (*Fxold*² - *xold* * *FFxold*) / (2 * *Fxold* - *xold* - *FFxold*)

xerr $\leftarrow$ |(xnew - xold)/xnew|

output *iter*, *xnew*, *xerr*

if *xerr* < ε then return

//method converges within *itmax* iterations//

else *xold* $\leftarrow$ *xnew*

end for

return

//method fails to converge in *itmax* iterations//

end procedure

Example 1. Find the roots of the equation $x^3 - 3x + 1 = 0$. Iterate until the relative error is less than $\varepsilon = 10^{-6}$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

- a. Cast the given equation into a form suitable for Wegstein's method, i.e., $x = F(x)$. Let

$$x = \frac{x^3 + 1}{3}$$

- b. Find initial estimates: see graph of $f(x) = x^3 - 3x + 1 = 0$

c. EASY procedures

```
procedure MAIN()
```

```
1: input  $x_0$ 
```

```
//initial estimate//
```

```
call WEGSTEIN( $10^{-6}$ , 20,  $x_0$ )
```

```
go to 1
```

```
end procedure
```

```
procedure F( $x$ )
```

```
return  $(x^3 + 1)/3$ 
```

```
end procedure
```

d. Results:

Original equation: $x^3 - 3x + 1 = 0$

Transformed equation: $x = \frac{x^3 + 1}{3}$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of Wegstein's method:

k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $	k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $	k	x_k	$\left \frac{x_k - x_{k-1}}{x_k} \right $
0	.5000000		0	-1.8000000		0	1.5000000	
1	.3451613	.4485981	1	-1.8990678	.0521665	1	1.5350706	.0228463
2	.3472961	.0061471	2	-1.8803619	.0099481	2	1.5321124	.0019308
3	.3472964	.0000006	3	-1.8793877	.0005183	3	1.5320889	.0000154
			4	-1.8793852	.0000013	4	1.5320889	.0000000
			5	-1.8793852	.0000000			

Example 2. For turbulent flow ($Re > 2000$) of a fluid in a smooth pipe, the relationship between the friction factor f and the Reynold's number Re is given by the equation

$$\sqrt{\frac{1}{f}} = 1.74 \ln(Re\sqrt{f}) - 0.40$$

Calculate f for $Re = 5000, 10000, 50000, 100000, 500000, 1000000$. Iterate until the relative error is less than $\varepsilon = 10^{-6}$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

a. Cast given equation in a form suitable for Wegstein's method, i.e., $f = F(f)$,

$$f = \frac{1}{[1.74 \ln(Re\sqrt{f}) - 0.40]^2}$$

b. Given Re , find an initial estimate. To this end, we use another empirical formula relating f and Re explicitly,

$$f = \frac{0.316}{Re^{0.25}} \quad (\text{Blassius's equation})$$

Blassius's equation gives an approximate value of f for a very smooth pipe for $3000 < Re < 100000$.

c. EASY procedures

```
procedure MAIN()
```

```
1: input  $Re$ 
```

```
//  $Re$  is assumed to be global //
```

```
 $f_0 \leftarrow 0.316/Re^{0.25}$ 
```

```
call WEGSTEIN( $10^{-6}$ , 20,  $f_0$ )
```

```
go to 1
```

```
end procedure
```

```

procedure F(f)
  return 1.0/(1.74 * ln(Re * sqrt(f)) - 0.40)^2
end procedure

```

d. Results:

Original equation: $\sqrt{\frac{1}{f}} = 1.74 \ln(Re \sqrt{f}) - 0.40$

Transformed equation: $f = \frac{1}{(1.74 \ln(Re \sqrt{f}) - 0.40)^2}$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of Wegstein's method:

k	f_k	$\left \frac{f_k - f_{k-1}}{f_k} \right $	k	f_k	$\left \frac{f_k - f_{k-1}}{f_k} \right $	k	f_k	$\left \frac{f_k - f_{k-1}}{f_k} \right $
0	.0375789	($Re = 5000$)	0	.0316000	($Re = 10000$)	0	.0177700	($Re = 100000$)
1	.0095356	2.9409245	1	.0078476	3.0267003	1	.0045330	2.9201082
2	.0093297	.0220679	2	.0077047	.0185538	2	.0044870	.0102679
3	.0093296	.0000072	3	.0077046	.0000042	3	.0044870	.0000007
4	.0093296	.0000000	4	.0077046	.0000000			

e. Summary of results for the friction problem: MoSS and Wegstein

- MosS

Reynolds number	Initial estimate	Final estimate	# of Iterations	Relative error
5000	.0375789	.0093296	9	.0000010
10000	.0316000	.0077046	9	.0000005
50000	.0211322	.0052110	8	.0000007
100000	.0177700	.0044870	8	.0000004
500000	.0118835	.0032814	8	.0000001
1000000	.0099928	.0029040	7	.0000009

- Wegstein's method

Reynolds number	Initial estimate	Final estimate	# of Iterations	Relative error
5000	.0375789	.0093296	4	.0000000
10000	.0316000	.0077046	4	.0000000
50000	.0211322	.0052110	4	.0000005
100000	.0177700	.0044870	3	.0000007
500000	.0118835	.0032814	3	.0000002
1000000	.0099928	.0029040	3	.0000001

1.7 Newton's Method

Let $\xi = x_k$ in the expression for ω , such that

$$\omega = \frac{1}{1 - F'(\xi)} = \frac{1}{1 - F'(x_k)}$$

Then, we have

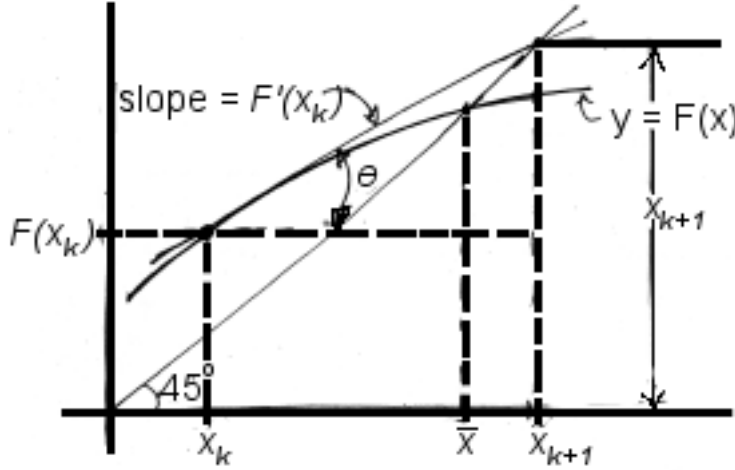
$$\begin{aligned} x_{k+1} &= x_k + \omega \cdot \delta_k \\ &= x_k + \frac{1}{1 - F'(x_k)} \cdot [F(x_k) - x_k] \end{aligned}$$

or

$$x_{k+1} = x_k + \frac{F(x_k) - x_k}{1 - F'(x_k)}, \quad (1.7)$$

which is Newton's formula for the form of the equation $x = F(x)$.

The graphical representation of this formula is shown below:



From the figure:

$$\tan \theta = F'(x_k) = \frac{x_{k+1} - F(x_k)}{x_{k+1} - x_k}$$

Hence, we have

$$\begin{aligned} x_{k+1} - F(x_k) &= x_{k+1} \cdot F'(x_k) - x_k \cdot F'(x_k) \\ x_{k+1} - x_{k+1} \cdot F'(x_k) &= F(x_k) - x_k \cdot F'(x_k) + x_k - x_k \\ x_{k+1}(1 - F'(x_k)) &= F(x_k) - x_k + x_k(1 - F'(x_k)) \end{aligned}$$

Finally

$$x_{k+1} = x_k + \frac{F(x_k) - x_k}{1 - F'(x_k)}$$

A simpler version of Newton's formula can be derived for the original equation $f(x) = 0$ by noting that

$$F(x) = x - f(x) \quad \text{and} \quad F'(x) = 1 - f'(x)$$

and substituting into (1.7) to yield

$$x_{k+1} = x_k + \frac{x_k - f(x_k) - x_k}{1 - [1 - f'(x_k)]}$$

or

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)},$$

which is Newton's iterative formula for the form of the equation $f(x) = 0$.

1.7.1 Taylor Series Expansion of a Function

Given a function $y = f(x)$, the Taylor series expansion of $f(x)$ about $x = a$ is

$$\begin{aligned} f(x) &= f(a) + \frac{f'(a)}{1!}(x-a)^1 + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \cdots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \cdots \\ &= \sum_{p=0}^{\infty} \frac{f^{(p)}(a)}{p!}(x-a)^p \end{aligned}$$

1.7.2 Derivation of Newton's Iterative Formula from the Taylor Series Expansion of $f(x)$

Expanding $f(x)$ about $x = x_k$ we have

$$f(x) = f(x_k) + \frac{f'(x_k)}{1!}(x - x_k)^1 + \dots \quad (1.8)$$

where $\dots$ means the remaining terms of the series (an infinite number of terms). Evaluating (1.8) at $x = \bar{x}$, we obtain

$$f(\bar{x}) = 0 = f(x_k) + \frac{f'(x_k)}{1!}(\bar{x} - x_k)^1 + \dots$$

Dropping the remaining terms of the series, we obtain the approximation

$$f(\bar{x}) = 0 \approx f(x_k) + \frac{f'(x_k)}{1!}(\bar{x} - x_k)^1 \quad (1.9)$$

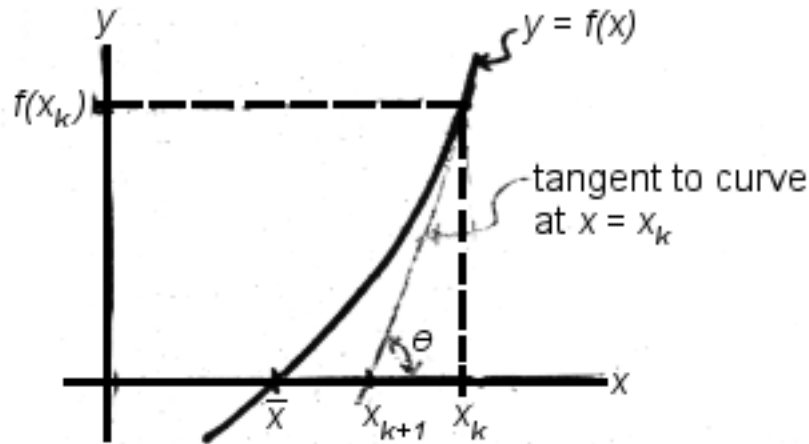
If (1.9) is to be of any computational use, the $\approx$ must be replaced by $=$. We can do this provided that we replace $\bar{x}$ by an estimate of $\bar{x}$, say x_{k+1} , thus:

$$0 = f(x_k) + \frac{f'(x_k)}{1!}(x_{k+1} - x_k)^1$$

Solving for x_{k+1} gives

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (1.10)$$

1.7.3 Graphical Representation of Newton's Formula



$$\begin{aligned} \tan \theta &= \frac{f'(x_k)}{f(x_k)} && \text{By construction} \\ &= \frac{f(x_k)}{x_k - x_{k+1}} && \text{By definition} \end{aligned}$$

Equating both expressions for $\tan \theta$ and solving for x_{k+1} yields Eq. (1.10).

1.7.4 Condition for Convergence of Newton's Method

From MoSS:

$$x_{k+1} = F(x_k) = x_k - \frac{f(x_k)}{f'(x_k)}$$

Newton's method is the MoSS if we take $F(x)$ to be

$$F(x) = x - \frac{f(x)}{f'(x)}$$

Hence, we have

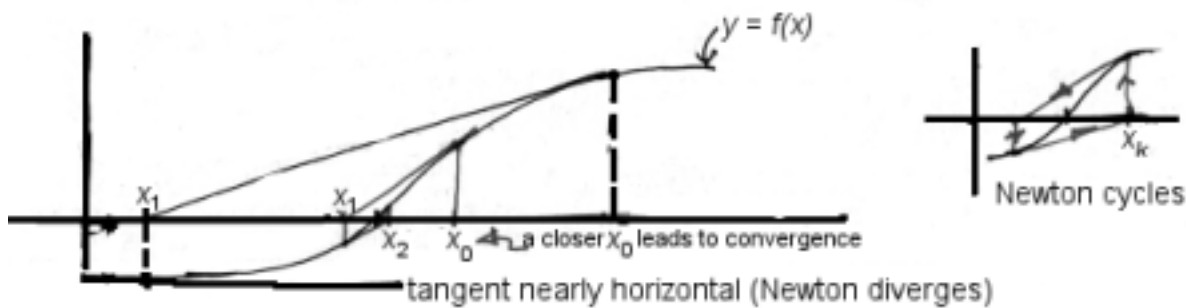
$$\begin{aligned} F'(x) &= 1 - \frac{f'(x)f'(x) - f(x)f''(x)}{[f'(x)]^2} \\ &= \frac{[f'(x)]^2 - [f'(x)]^2 + f(x)f''(x)}{[f'(x)]^2} \\ &= \frac{f(x)f''(x)}{[f'(x)]^2} \end{aligned}$$

and the condition for convergence of Newton's method is

$$|F'(x)| = \left| \frac{f(x)f''(x)}{[f'(x)]^2} \right| < 1$$

Qualitatively:

1. x_0 should be sufficiently close to $\bar{x}$
2. $f'(x_k)$ should not be close to zero for $k = 0, 1, 2, \dots$



Example: Calculate $\sin^{-1} 0.52$ using $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| < 0.001$

Let $x = \sin^{-1} 0.52$ or $\sin x = 0.52$. Then:

$$\begin{aligned} f(x) &= \sin x - 0.52 = 0 \\ f'(x) &= \cos x \end{aligned}$$

1st iteration: We note that $\sin^{-1} 0.50 = 30^\circ = \frac{\pi}{6}$. Taking $x_0 = \frac{\pi}{6} = 0.523599$:

$$\begin{aligned} f(x_0) &= \sin \frac{\pi}{6} - 0.52 = -0.02 \\ f'(x_0) &= \cos \frac{\pi}{6} = 0.866025 \end{aligned}$$

$$\begin{aligned}
x_1 &= x_0 - \frac{f(x_0)}{f'(x_0)} = 0.523599 - \frac{-0.020000}{0.866025} = 0.523599 + 0.023094 \\
&= 0.546693 \\
\left| \frac{x_1 - x_0}{x_1} \right| &= \frac{0.023094}{0.546693} = 0.042243
\end{aligned}$$

2nd iteration

$$\begin{aligned}
f(x_1) &= 0.000135 \\
f'(x_1) &= 0.854248 \\
x_2 &= x_1 - \frac{f(x_1)}{f'(x_1)} = 0.546535 \\
\left| \frac{x_2 - x_1}{x_2} \right| &= 0.000289 < 0.001 \\
\sin^{-1} 0.52 &\approx 0.546535
\end{aligned}$$

1.7.5 Summary

Form of the equation: $f(x) = 0$

Iterative formula: $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$

Convergence criterion: $\left| \frac{f(x)f''(x)}{[f'(x)]^2} \right| < 1$

Criteria for terminating the iterations:

- a. $|f(x_k)| < \varepsilon$ or
- b. Absolute error criterion: $|x_{k+1} - x_k| < \varepsilon$ or
- c. Relative error criterion: $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| < \varepsilon$ or
- d. others and
- e. A prescribed maximum number of iterations, $itmax$

1.7.6 EASY Implementation

procedure NEWTON($\varepsilon, itmax, x$)

$iter \leftarrow 0$

$fx \leftarrow f(x)$

 //function call to evaluate $f(x_0)$ //

 output $iter, x, fx$, 'Initial estimate'

 for $iter \leftarrow 1$ to $itmax$ do

$dfx \leftarrow df(x)$

 //function call to evaluate $f'(x_k), k = 0, 1, 2, \dots //$

 if $dfx = 0$ then return

 //zero slope; terminate iterations //

$x \leftarrow x - fx/dfx$

$fx \leftarrow f(x)$

 //function call to evaluate $f(x_k), k = 1, 2, \dots //$

 output $iter, x, fx$

 if $|fx| < \varepsilon$ then return

 //method converges within $itmax$ iterations //

 end for

 return

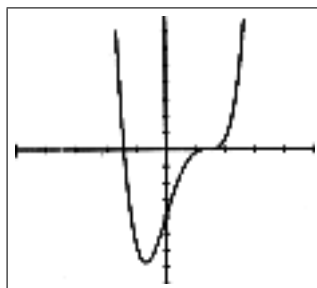
 //method fails to converge in $itmax$ iterations //

end procedure

Example 1. Find the roots of the equation $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$. Iterate until $|f(x_k)| < \varepsilon$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

- a. Find initial estimates



From the plot of $f(x) = 0$, we see that there is a distinct negative root near -1.5 and three equal positive roots near 1.5 .

- b. EASY procedures

```

procedure MAIN()
1: input x0                                     //initial estimate//
   call NEWTON( $10^{-6}$ , 20, x0)
   go to 1
end procedure

procedure f(x)
   return  $x^4 - 2 * \sqrt{2} * x^3 + 4 * \sqrt{2} * x - 4$ 
end procedure

procedure df(x)
   return  $4 * x^3 - 6 * \sqrt{2} * x^2 + 4 * \sqrt{2}$ 
end procedure

```

- c. Results

Given equation: $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of Newton's method:

k	x_k	$ f(x_k) $	k	x_k	$ f(x_k) $
0	-1.5000000	2.1231602	0	1.5000000	.0018398
1	-1.4211748	.1586797	1	1.4716824	.0005477
2	-1.4142645	.0011517	2	1.4526524	.0001628
3	-1.4142136	.0000001	3	1.4398965	.0000484
			4	1.4313611	.0000143
			5	1.4256567	.0000043
			6	1.4218475	.0000013
			7	1.4193051	.0000004

The true roots are $\bar{x}_1 = -\sqrt{2} = -1.414213562\dots$ and $\bar{x}_2 = \bar{x}_3 = \bar{x}_4 = \sqrt{2}$. Note the rapid convergence (quadratic) of Newton's method in finding $\bar{x}_1$ and the accuracy of the result. In contrast, the method exhibits slow convergence (linear) in finding the multiple root. Likewise, the final estimate is accurate only to three digits, a tolerance of 10^{-6} notwithstanding. This is because the function is *tangent* to the x -axis at a multiple root (crosses the axis if the multiplicity is odd, but does not, otherwise). A very small tolerance must be used to get more nearly accurate estimates.

Given equation: $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$

$itmax = 20, \varepsilon = 10^{-12}$

Iterations of Newton's method:

k	x_k	$ f(x_k) $
0	1.5000000	.1839828D - 02
1	1.4716824	.5477439D - 03
2	1.4526524	.1628243D - 03
3	1.4398965	.4835084D - 04
4	1.4313611	.1434750D - 04
$\vdots$		
15	1.4144124	.2223609D - 10
16	1.4143461	.6588547D - 11
17	1.4143019	.1952177D - 11
18	1.4142725	.5784258D - 12

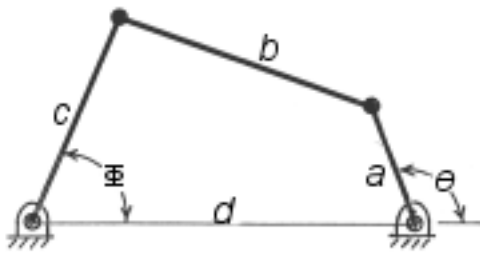
Given equation: $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$

$itmax = 30, \varepsilon = 10^{-16}$

Iterations of Newton's method:

k	x_k	$ f(x_k) $
0	1.5000000	.1839828D - 02
1	1.4716824	.5477439D - 03
2	1.4526524	.1628243D - 03
3	1.4398965	.4835084D - 04
4	1.4313611	.1434750D - 04
$\vdots$		
23	1.4142213	.1320558D - 14
24	1.4142187	.3911801D - 15
25	1.4142170	.1162265D - 15
26	1.4142159	.3426079D - 16

Example 2. Shown below is the schematic diagram of a 4-bar mechanism.



θ = input angle
 Φ = output angle
 a = length of input crank
 b = length of coupler link
 c = length of output crank
 d = length of fixed link

The relationship between the input angle θ and the output angle Φ is given by Freudenstein's equation

$$R_1 \cos \theta - R_2 \cos \Phi + R_3 - \cos(\theta - \Phi) = 0,$$

where

$$\begin{aligned}
 R_1 &= \frac{d}{c} \\
 R_2 &= \frac{d}{a} \\
 R_3 &= \frac{a^2 - b^2 + c^2 + d^2}{2ac}
 \end{aligned}$$

For $b = c = d = 2a$, the input crank describes a full circle while the output crank oscillates. It is desired to generate a table of θ vs Φ values for the full range of motion of the mechanism, with θ varying in increments of 10° .

Solution: We will use Newton's method with the following termination criteria: $|f(x)| < 10^{-6}$, $itmax = 20$.

- a. Initial estimates: we need to generate 36 initial estimates of Φ , one for each value of θ . For $\theta = 0^\circ$, we draw the mechanism (more or less to scale) and measure Φ , say with a protractor. We find that Φ is approximately 40° . For $\theta = 10^\circ, 20^\circ, 30^\circ, \dots$, we will use as Φ_0 the *final* estimate of Φ that we obtained for $\theta = 0^\circ, 10^\circ, 20^\circ, \dots$, respectively.

- b. EASY procedures

```

procedure MAIN()                                // R1, R2, R3 and  $\theta$  are assumed to be global variables //
  input a, b, c, d,  $\Phi$                           // 1, 2, 2, 2, 40 //
  R1  $\leftarrow d/c$ 
  R2  $\leftarrow d/a$ 
  R3  $\leftarrow (a^2 - b^2 + c^2 + d^2)/(2 * a * c)$ 
   $\Phi \leftarrow \Phi * 0.0174533$ 
  for  $\theta \leftarrow 0$  to 360 by 10 do
     $\theta \leftarrow \theta * 0.0174533$ 
    call NEWTON( $10^{-6}$ , 20,  $\Phi$ )
  end for
end procedure

procedure f( $\Phi$ )
  return R1 * cos( $\theta$ ) - R2 * cos( $\Phi$ ) + R3 - cos( $\theta - \Phi$ )
end procedure

procedure df( $\Phi$ )
  return R2 * sin( $\Phi$ ) - sin( $\theta - \Phi$ )
end procedure

```

- c. Results

Given equation: $R_1 \cos \theta - R_2 \cos \Phi + R_3 - \cos(\theta - \Phi) = 0$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of Newton's method:

k	Φ_k	$ f(\Phi_k) $
0	.6981317	.0481333($\theta = 0^\circ$)
1	.7230924	.0007109
2	.7227343	.0000001

k	Φ_k	$ f(\Phi_k) $
0	.7227343	.1186553($\theta = 10^\circ$)
1	.7870801	.0047886
2	.7846756	.0000065
3	.7846724	.0000000

k	Φ_k	$ f(\Phi_k) $
0	.7846724	.1321611($\theta = 20^\circ$)
1	.8566891	.0059042
2	.8537332	.0000095
3	.8537284	.0000000

k	Φ_k	$ f(\Phi_k) $
0	.8537284	.1443298($\theta = 30^\circ$)
1	.9325267	.0068645
2	.9291011	.0000124
3	.9290949	.0000000

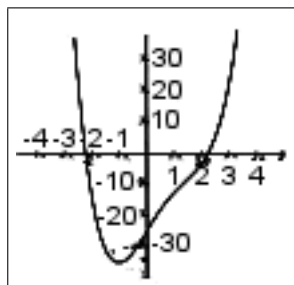
d. Summary of results for Freudenstein's equation: Newton's method

θ , degrees	Φ , degrees	$ f(\Phi) $	Iterations
0	41.4096300	.0000001	2
10	44.9584200	.0000000	3
20	48.9150400	.0000000	3
30	53.2332200	.0000000	3
40	57.8526700	.0000000	3
50	62.7013200	.0000000	3
60	67.6969800	.0000000	3
70	72.7474100	.0000000	3
80	77.7480900	.0000000	3
90	82.5772100	.0000000	3
100	87.0869300	.0000000	3
110	91.0901000	.0000000	3
120	94.3411000	.0000003	2
130	96.5106600	.0000000	2
140	97.1608200	.0000000	2
150	95.7450900	.0000000	2
160	91.7057300	.0000001	2
170	84.7856700	.0000002	2
180	75.5225100	.0000004	2
190	65.3727300	.0000007	2
200	55.9497900	.0000008	2
210	48.1571700	.0000008	2
220	42.1293600	.0000006	2
230	37.6277500	.0000004	2
240	34.3411000	.0000002	2
250	32.0036100	.0000001	2
260	30.4179500	.0000000	2
270	29.4471100	.0000000	2
280	29.0006900	.0000000	2
290	29.0229700	.0000001	1
300	29.4837700	.0000000	2
310	30.3715600	.0000000	2
320	31.6876900	.0000001	2
330	33.4410400	.0000003	2
340	35.6425100	.0000008	2
350	38.2992900	.0000000	3
360	41.4096200	.0000000	3

Example 3. Find the roots of the equation $x^4 - 2\sqrt{2}x^3 + 10\sqrt{2}x - 25 = 0$. iterate until $|f(x_k)| < 10^{-6}$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

- a. Find initial estimates.



From the plot of $f(x) = 0$, we see that there is a negative root near -2.1 and a positive root near 2.1 . The two other roots are complex. Assume that we obtain initial estimates for the complex roots by factoring out $(x + 2)$ and $(x - 2)$ to obtain a quadratic equation whose roots are found from the quadratic formula. These turn out to be $1.5 \pm 1.3i$ if we replace $2\sqrt{2}$ and $10\sqrt{2}$ in the equation by 3 and 14 respectively (just to simplify the computations).

b. EASY procedures

procedure NEWTON can be used to find all the roots of the given equation by simply declaring x , fx and dfx to be complex. Note that $|fx|$ is real, so the test

if $|fx| < \varepsilon$ then return

where ε is real is valid.

procedure MAIN()

1: input x_0

// initial estimate must be a complex number //

call NEWTON(10^{-6} , 20, x_0) go to 1

end procedure

procedure $f(x)$

// function must be declared as complex //

return $x^4 - 2 * \sqrt{2} * x^3 + 10 * \sqrt{2} * x - 25$

end procedure

procedure $df(x)$

// function must be declared as complex //

return $4 * x^3 - 6 * \sqrt{2} * x^2 + 10 * \sqrt{2}$

end procedure

c. Results

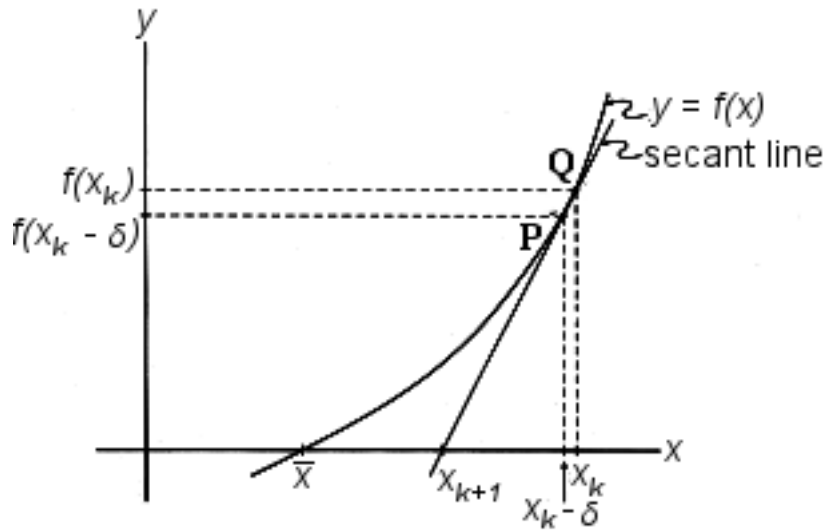
k	x_k	$f(x_k)$	$ f(x_k) $
0	$-2.1000000 + .0000000i$	$-9.0563212 + .0000000i$	9.0563212
1	$-2.2501331 + .0000000i$	$1.0365531 + .0000000i$	1.0365531
2	$-2.2361991 + .0000000i$	$.0095732 + .0000000i$	.0095732
3	$-2.2360680 + .0000000i$	$.0000008 + .0000000i$	.0000008

k	x_k	$f(x_k)$	$ f(x_k) $
0	$2.1000000 + .0000000i$	$-2.0474788 + .0000000i$	2.0474788
1	$2.2487340 + .0000000i$	$.2099738 + .0000000i$	.2099738
2	$2.2361753 + .0000000i$	$.0017637 + .0000000i$	.0017637
3	$2.2360680 + .0000000i$	$.0000001 + .0000000i$	.0000001

k	x_k	$f(x_k)$	$ f(x_k) $
0	$1.5000000 + 1.3000000i$	$-6.7189498 + 4.1473827i$	7.8958894
1	$1.2811528 + 2.0478859i$	$11.7424264 + -2.0555213i$	11.9209792
2	$1.3648547 + 1.7872348i$	$2.0749252 + .1814712i$	2.0828458
3	$1.4098347 + 1.7327260i$	$.0880942 + .0799369i$	.1189559
4	$1.4142084 + 1.7320346i$	$-.0002478 + .0003821i$	.0004554
5	$1.4142136 + 1.7320508i$	$.0000000 + .0000000i$	.0000000

1.7.7 A Variation on Newton's Method

If $f(x)$ is such that evaluating $f'(x_k)$ as $k = 0, 1, 2, \dots$ is too cumbersome, then a *secant*, in lieu of a tangent, line may be used to find x_{k+1} .



The iterative formula now becomes

$$x_{k+1} = x_k - \frac{f(x_k)}{s}$$

where

$$s = \frac{f(x_k) - f(x_k - \delta)}{\delta}$$

for some suitable δ .

Example:

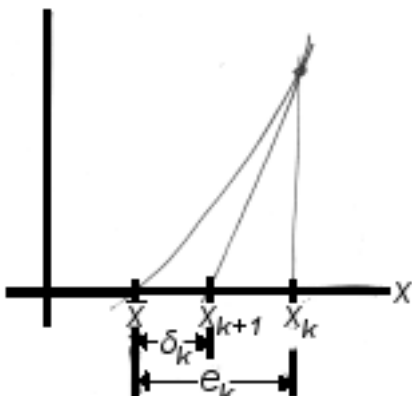
$$f(e) = \frac{\pi\phi}{360} - \sqrt{\left(\frac{1+e\cos\theta}{1-e^2}\right)^3} \left[2\tan^{-1}\left(\sqrt{\frac{1-e}{1+e}}\tan\frac{\theta}{2}\right) - \frac{e\sqrt{1-e^2}\sin\theta}{1+e\cos\theta} \right] = 0$$

1.8 Richmond's Method

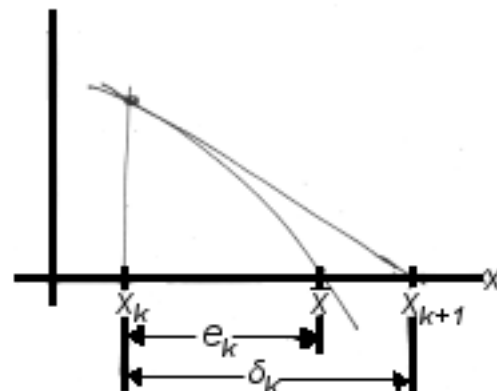
1.8.1 Motivation

Consider applying a relaxation factor ω to Newton's method, as:

$$x_{k+1} = x_k - \omega \frac{f(x_k)}{f'(x_k)}$$



Newton's method undershoots the root:
Apply $\omega > 1$



Newton's method overshoots the root:
Apply $\omega < 1$

1.8.2 Richmond's method: using Taylor Series Expansion of $f(x)$ About $x = x_k$

$$\begin{aligned}
 f(x) &= f(x_k) + \frac{f'(x_k)}{1!}(x - x_k)^1 + \frac{f''(x_k)}{2!}(x - x_k)^2 + \dots \\
 f(\bar{x}) = 0 &= f(x_k) + \frac{f'(x_k)}{1!}(\bar{x} - x_k)^1 + \frac{f''(x_k)}{2!}(\bar{x} - x_k)^2 + \dots \\
 0 &= f(x_k) + \frac{f'(x_k)}{1!}(x_{k+1} - x_k)^1 + \frac{f''(x_k)}{2!}(x_{k+1} - x_k)^2 \\
 0 &= f(x_k) + (x_{k+1} - x_k) \left[f'(x_k) + \frac{1}{2} f''(x_k)(x_{k+1} - x_k) \right]
 \end{aligned}$$

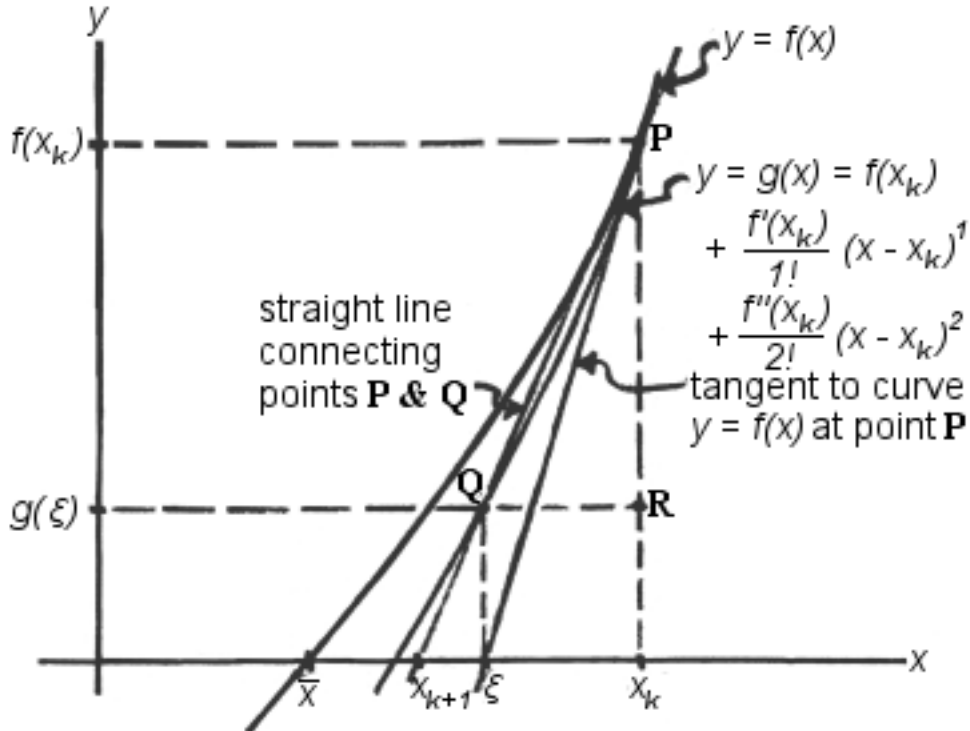
Using the relation $x_{k+1} - x_k = -\frac{f(x_k)}{f'(x_k)}$ from Newton's method to replace the expression within the brackets, we have:

$$\begin{aligned}
 x_{k+1} &= x_k - \frac{f(x_k)}{f'(x_k) + \frac{1}{2} f''(x_k) \left[-\frac{f(x_k)}{f'(x_k)} \right]} \\
 x_{k+1} &= x_k - \frac{2f(x_k)f'(x_k)}{2[f'(x_k)]^2 - f(x_k)f''(x_k)} \quad \text{Richmond's iterative formula} \\
 &= x_k - \frac{2[f'(x_k)]^2}{2[f'(x_k)]^2 - f(x_k)f''(x_k)} \cdot \frac{f(x_k)}{f'(x_k)}
 \end{aligned}$$

The expression for ω is

$$\omega = \frac{2[f'(x_k)]^2}{2[f'(x_k)]^2 - f(x_k)f''(x_k)} = \frac{1}{1 - \frac{f(x_k)f''(x_k)}{2[f'(x_k)]^2}}$$

1.8.3 Graphical Interpretation



Let ξ be the estimate of $\bar{x}$ predicted by Newton's formula. Then

$$\xi = x_k - \frac{f(x_k)}{f'(x_k)}$$

from which

$$\xi - x_k = -\frac{f(x_k)}{f'(x_k)}$$

From the figure above,

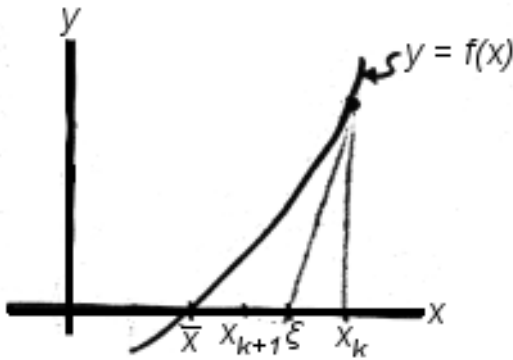
$$\begin{aligned} PR &= f(x_k) - g(\xi) \\ &= f(x_k) - \left[f(x_k) + \frac{f'(x_k)}{1!}(\xi - x_k) + \frac{f''(x_k)}{2!}(\xi - x_k)^2 \right] \\ &= - \left[f'(x_k)(\xi - x_k) + \frac{1}{2}f''(x_k)(\xi - x_k)^2 \right] \end{aligned}$$

By similar triangles

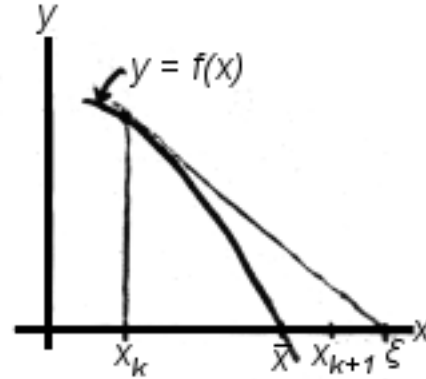
$$\begin{aligned} \frac{x_k - x_{k+1}}{f(x_k)} = \frac{QR}{PR} &= \frac{x_k - \xi}{- \left[f'(x_k)(\xi - x_k) + \frac{1}{2}f''(x_k)(\xi - x_k)^2 \right]} \\ \frac{x_k - x_{k+1}}{f(x_k)} &= \frac{-(\xi - x_k)}{- \left[f'(x_k)(\xi - x_k) + \frac{1}{2}f''(x_k)(\xi - x_k)^2 \right]} \\ x_k - x_{k+1} &= \frac{f(x_k)}{f'(x_k) + \frac{1}{2}f''(x_k) \left[-\frac{f(x_k)}{f'(x_k)} \right]} \end{aligned}$$

Solving for x_{k+1} ,

$$x_{k+1} = x_k - \frac{2f(x_k)f'(x_k)}{2[f'(x_k)]^2 - f(x_k)f''(x_k)}$$



Over-relaxation: $\omega > 1$
 $f(x_k)f''(x_k) > 0$



Under-relaxation: $\omega < 1$
 $f(x_k)f''(x_k) < 0$

Example: Find the positive root of $2 \cos x - e^x = 0$. Use the criterion $f(x_k) < 0.001$ to stop the iterations.

$$\begin{aligned} f(x) &= 2 \cos x - e^x = 0 \\ f'(x) &= -2 \sin x - e^x \\ f''(x) &= -2 \cos x - e^x \end{aligned}$$

1st iteration: Take $x_0 = 0.50$

$$\begin{aligned}
 f(x_0) &= 2 \cos(0.50) - e^{0.50} = 0.106444 \\
 f'(x_0) &= -2 \sin(0.50) - e^{0.50} = -2.607572 \\
 f''(x_0) &= -2 \cos(0.50) - e^{0.50} = -3.403886 \\
 x_1 &= x_0 - \frac{2f(x_0)f'(x_0)}{2[f'(x_0)]^2 - f(x_0)f''(x_0)} \\
 &= 0.50 - \frac{2(0.106444)(-2.607572)}{2(-2.607572)^2 - (0.106444)(-3.403886)} = 0.539762
 \end{aligned}$$

2nd iteration

$$\begin{aligned}
 f(x_1) &= 2 \cos(0.539762) - e^{0.539762} = 0.000064 < 0.001 \\
 \bar{x} &\approx 0.539762
 \end{aligned}$$

1.8.4 Summary

Form of the equation: $f(x) = 0$

Iterative formula: $x_{k+1} = x_k - \frac{2f(x_k)f'(x_k)}{2[f'(x_k)]^2 - f(x_k)f''(x_k)}$

Convergence criterion: never mind

Criteria for terminating the iterations:

- a. $|f(x_k)| < \varepsilon$ or
- b. Absolute error criterion: $|x_{k+1} - x_k| < \varepsilon$ or
- c. Relative error criterion: $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| < \varepsilon$ or
- d. others and
- e. A prescribed maximum number of iterations, $itmax$

1.8.5 EASY Implementation

procedure RICHMOND($\varepsilon, itmax, x$)

```

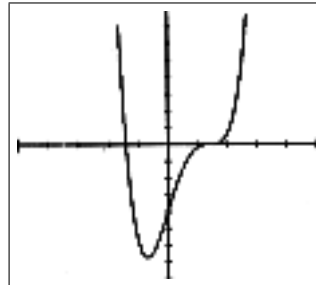
    iter ← 0
    fx ← f(x) //function call to evaluate f(x_0)//
    output iter, x, fx, 'Initial estimate'
    for iter ← 1 to itmax do
        dfx ← df(x) //function call to evaluate f'(x_k), k = 0, 1, 2, ...//
        ddfx ← ddf(x) //function call to evaluate f''(x_k), k = 0, 1, 2, ...//
        x ← x - (2 * fx * dfx) / (2 * dfx * dfx - fx * ddfx)
        fx ← f(x) //function call to evaluate f(x_k), k = 1, 2, ...//
        output iter, x, fx
        if |fx| < ε then return //method converges within itmax iterations//
    end for
    return //method fails to converge in itmax iterations//
end procedure

```

Example 1. Find the roots of the equation $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$. Iterate until $|f(x_k)| < \varepsilon$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

- a. Find initial estimates



From the plot of $f(x) = 0$, we see that there is a distinct negative root near -1.5 and three equal positive roots near 1.5 .

- b. EASY procedures

```
procedure MAIN()
```

```
1: input x0
```

```
    call RICHMOND( $10^{-6}$ , 20, x0)
```

```
    go to 1
```

```
end procedure
```

//initial estimate//

```
procedure f(x)
```

```
    return  $x^4 - 2 * \sqrt{2} * x^3 + 4 * \sqrt{2} * x - 4$ 
```

```
end procedure
```

```
procedure df(x)
```

```
    return  $4 * x^3 - 6 * \sqrt{2} * x^2 + 4 * \sqrt{2}$ 
```

```
end procedure
```

```
procedure ddf(x)
```

```
    return  $12 * x^2 - 12 * \sqrt{2} * x$ 
```

```
end procedure
```

- c. Results

Given equation: $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of Richmond's method:

k	x_k	$ f(x_k) $
0	-1.5000000	2.1231602
1	-1.4146215	.0092335
2	-1.4142136	.0000000

k	x_k	$ f(x_k) $
0	1.5000000	.0018398
1	1.4573202	.0002300
2	1.4358212	.0000288
3	1.4250311	.0000036
4	1.4196258	.0000004

Given equation: $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$

$itmax = 20, \varepsilon = 10^{-12}$

Iterations of Richmond's method:

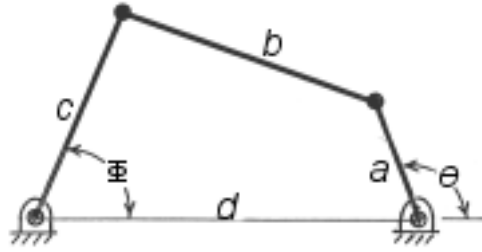
k	x_k	$ f(x_k) $
0	1.5000000	.1839828D - 02
1	1.4573202	.2300106D - 03
2	1.4358212	.2875238D - 04
3	1.4250311	.3594082D - 05
4	1.4196258	.4492613D - 06
5	1.4169205	.5615769D - 07
6	1.4155673	.7019713D - 08
7	1.4148905	.8774641D - 09
8	1.4145520	.1096831D - 09
9	1.4143828	.1371091D - 10
10	1.4142982	.1713973D - 11
11	1.4142559	.2143927D - 12

Given equation: $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$

$itmax = 30, \varepsilon = 10^{-16}$

Iterations of Richmond's method:

k	x_k	$ f(x_k) $
0	1.5000000	.1839828D - 02
1	1.4573202	.2300106D - 03
2	1.4358212	.2875238D - 04
3	1.4250311	.3594082D - 05
4	1.4196258	.4492613D - 06
5	1.4169205	.5615769D - 07
6	1.4155673	.7019713D - 08
7	1.4148905	.8774641D - 09
8	1.4145520	.1096831D - 09
9	1.4143828	.1371091D - 10
10	1.4142982	.1713973D - 11
11	1.4142559	.2143927D - 12
12	1.4142346	.2621514D - 13
13	1.4142242	.3596949D - 14
14	1.4142186	.1565588D - 15
15	1.4142192	.1131907D - 15
16	1.4142196	.7190429D - 15
17	1.4142158	.4978656D - 15
18	1.4142186	.6557255D - 15
19	1.4142108	.5139118D - 15
20	1.4142087	.2053479D - 15
21	1.4142100	.2025290D - 15
22	1.4142142	.2320193D - 15
23	1.4142148	.8456777D - 16

Example 2. Freudenstein's equation

Solution: We will use the Richmond method with the following termination criteria: $|f(x)| < 10^{-6}$, $itmax = 20$.

a. Initial estimates: Same as in Newton's method

b. EASY procedures

```

procedure MAIN()                                     // R1, R2, R3 and θ are assumed to be global variables //
  input a, b, c, d, Φ                                  // 1, 2, 2, 2, 40 //
  R1 ← d/c
  R2 ← d/a
  R3 ← (a2 - b2 + c2 + d2)/(2 * a * c)
  Φ ← Φ * 0.0174533
  for θ ← 0 to 360 by 10 do
    θ ← θ * 0.0174533
    call RICHMOND(10-6, 20, Φ)
  end for
end procedure

procedure f(Φ)
  return R1 * cos(θ) - R2 * cos(Φ) + R3 - cos(θ - Φ)
end procedure

procedure df(Φ)
  return R2 * sin(Φ) - sin(θ - Φ)
end procedure

procedure ddf(Φ)
  return R2 * cos(Φ) + cos(θ - Φ)
end procedure

```

c. Results

Given equation: $R_1 \cos \theta - R_2 \cos \Phi + R_3 - \cos(\theta - \Phi) = 0$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of Richmond's method:

k	Φ_k	$ f(\Phi_k) $
0	.6981317	.0481333($\theta = 0^\circ$)
1	.7227266	.0000152
2	.7227342	.0000000

k	Φ_k	$ f(\Phi_k) $
0	.7227342	.1186555($\theta = 10^\circ$)
1	.7845422	.0002586
2	.7846724	.0000000

k	Φ_k	$ f(\Phi_k) $
0	.7846724	.1321613($\theta = 20^\circ$)
1	.8535511	.0003530
2	.8537284	.0000000

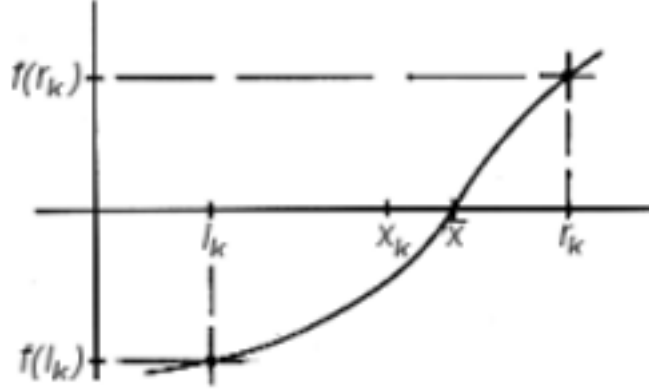
k	Φ_k	$ f(\Phi_k) $
0	.8537284	.1443299($\theta = 30^\circ$)
1	.9288731	.0004428
2	.9290949	.0000000

d. Summary of results for Freudenstein's equation: Richmond's method

θ , degrees	Φ , degrees	$ f(\Phi) $	Iterations
0	41.4096200	.0000000	2
10	44.9584200	.0000000	2
20	48.9150400	.0000000	2
30	53.2332200	.0000000	2
40	57.8526700	.0000000	2
50	62.7013200	.0000000	2
60	67.6969800	.0000000	2
70	72.7474100	.0000000	2
80	77.7480900	.0000000	2
90	82.5772100	.0000000	2
100	87.0869300	.0000000	2
110	91.0901000	.0000000	2
120	94.3410900	.0000000	2
130	96.5106600	.0000000	2
140	97.1608000	.0000004	1
150	95.7450900	.0000000	2
160	91.7057300	.0000000	2
170	84.7856500	.0000000	2
180	75.5224900	.0000000	2
190	65.3726900	.0000000	2
200	55.9497500	.0000000	2
210	48.1571400	.0000000	2
220	42.1293300	.0000000	2
230	37.6277300	.0000000	2
240	34.3410900	.0000000	2
250	32.0036100	.0000000	2
260	30.4179500	.0000000	2
270	29.4471100	.0000000	2
280	29.0007000	.0000003	1
290	29.0229700	.0000000	1
300	29.4837600	.0000004	1
310	30.3715600	.0000000	2
320	31.6876900	.0000000	2
330	33.4410300	.0000000	2
340	35.6424900	.0000000	2
350	38.2992900	.0000000	2
360	41.4096200	.0000000	2

1.9 Half-Interval Method

1.9.1 Algorithm



1. Choose l_k and r_k , $k = 0$, such that $f(l_k)f(r_k) < 0$.
2. Calculate

$$x_k = \frac{l_k + r_k}{2} = l_k + \frac{r_k - l_k}{2}$$

On a computer, in which numbers are represented as floating-point numbers with fixed precision, the midpoint of the interval should be calculated as the rightmost formula.

Example: Suppose we have a computer which implements four decimal digit chopped floating-point arithmetic.

$$l_k = 0.8763 \quad r_k = 0.8766$$

$$\begin{aligned} x &= \frac{l_k + r_k}{2} = \frac{0.8763 + 0.8766}{2} = \frac{1.7529}{2} \xrightarrow{\text{chopped}} \frac{1.752}{2} \\ &= 0.8760 \text{ which is outside the interval} \\ x &= l_k + \frac{r_k - l_k}{2} = 0.8763 + \frac{0.8766 - 0.8763}{2} = 0.8763 + \frac{0.0003}{2} = 0.8763 + 0.00015 \\ &\xrightarrow{\text{chopped}} 0.8763 + 0.0001 = 0.8764 \text{ as it should be} \end{aligned}$$

“The general principle is that it is best to arrange formulas so that they express the desired quantity as a small correction to a good approximation.”

Forsythe, *et al.*, *Computer Methods for Mathematical Computations*,
Prentice-Hall, 1977, p. 162

3. If $f(l_0)f(x_k) > 0$, set $l_{k+1} = x_k$ and $r_{k+1} = r_k$; else, set $l_{k+1} = l_k$ and $r_{k+1} = x_k$

The test $f(l_0)f(x_k) > 0$ may fail due to underflow. For example, if $f(x_k) = 10^{-30}$ and $f(l_0) = 10^{-20}$ (in single precision) then an underflow will occur. To avoid this, we can define a multiplier as follows:

$$\begin{aligned} \text{if } f(l_0) < 0 \text{ then } fL0 &\leftarrow -1 \\ \text{else } fL0 &\leftarrow +1 \end{aligned}$$

The test above then becomes $fL0 \cdot f(x_k) > 0 \dots$

4. Repeat steps 2 and 3 until

a. Absolute error criterion: $|r_{k+1} - l_{k+1}| < \varepsilon$ or

If the absolute error criterion is used, it is possible to determine, beforehand, the number of iterations needed to reduce the initial interval $r_0 - l_0$ to the specified tolerance ε .

Let $\delta_0 = r_0 - l_0$ = original interval containing $\bar{x}$. Then:

$$\begin{aligned}\delta_1 &= \frac{\delta_0}{2} = \text{interval after the 1st iteration} \\ \delta_2 &= \frac{\delta_1}{2} = \frac{\delta_0}{4} = \text{interval after the 2nd iteration} \\ &\vdots \\ \delta_k &= \frac{\delta_0}{2^k} = \text{interval after the } k\text{th iteration}\end{aligned}$$

Now, let n be the number of iterations needed to reduce δ_0 to $\leq \varepsilon$; then

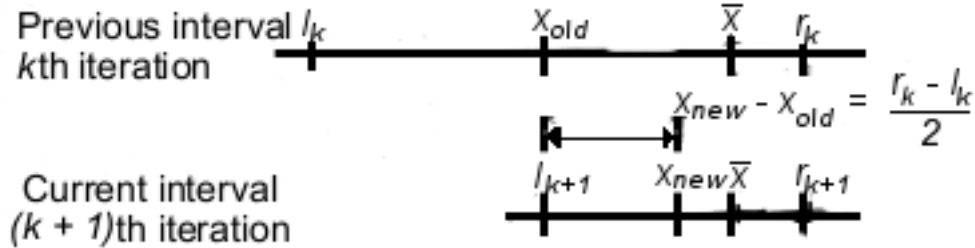
$$\frac{\delta_0}{2^n} \leq \varepsilon \implies 2^n \geq \frac{\delta_0}{\varepsilon} \implies \log 2^n \geq \log \frac{\delta_0}{\varepsilon} \implies n = \left\lceil \frac{\log(\delta_0/\varepsilon)}{\log 2} \right\rceil$$

Example: Let $\delta_0 = 1.0$, $\varepsilon = 10^{-6}$

$$n = \left\lceil \frac{\log(\delta_0/\varepsilon)}{\log 2} \right\rceil = \left\lceil \frac{\log(1.0/10^{-6})}{\log 2} \right\rceil = \left\lceil \frac{6}{0.3010} \right\rceil = \lceil 19.93 \rceil = 20$$

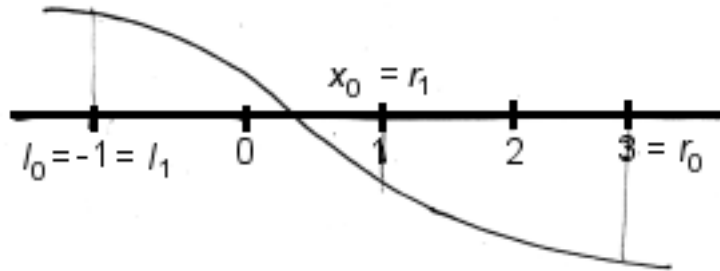
b. Relative error criterion: $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| = \left| \frac{r_{k+1} - l_{k+1}}{r_{k+1} + l_{k+1}} \right| < \varepsilon$ or

The relative error $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right|$ can be expressed in terms of the current interval limits l_{k+1} and r_{k+1} , thus obviating the need to keep x_k .



$$\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| = \left| \frac{\frac{r_{k+1} - l_{k+1}}{2}}{\frac{r_{k+1} + l_{k+1}}{2}} \right| = \left| \frac{r_{k+1} - l_{k+1}}{r_{k+1} + l_{k+1}} \right|$$

Note that $r_{k+1} + l_{k+1}$ may become zero; this means that a zero test must be included to avoid a floating-point divide exception.



c. $|f(x_k)| < \varepsilon$ or
d. the number of iterations $> itmax$

1.9.2 EASY Implementation

```

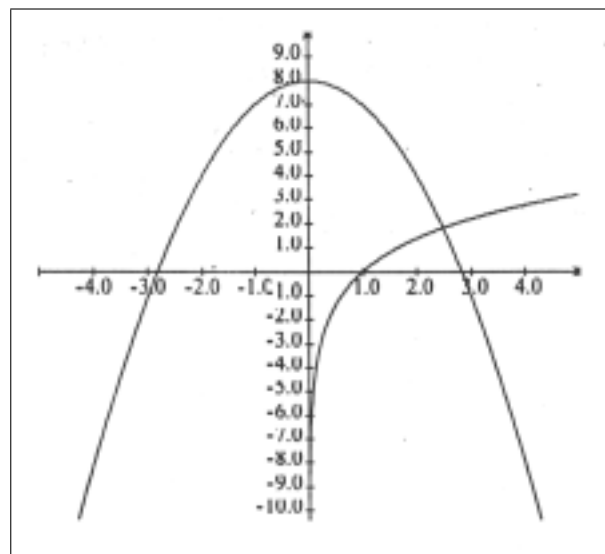
procedure HALF_INTERVAL( $\varepsilon$ ,  $itmax$ ,  $L$ ,  $R$ )
   $fL0 \leftarrow f(L)$ 
  for  $iter \leftarrow 0$  to  $itmax$  do
    //Computation//
     $xerr \leftarrow R - L$ 
     $x \leftarrow L + xerr/2$ 
    output  $iter, L, R, x, xerr$ 
    //Test for termination//
    if  $xerr < \varepsilon$  then return
    //Updating//
     $fx \leftarrow f(x)$ 
    if  $fL0 * fx > 0$  then  $L \leftarrow x$ 
      else  $R \leftarrow x$ 
  end for
  return //  $\varepsilon$  not satisfied in  $itmax$  iterations//
end procedure

```

Example 1. Find the root of the equation $2 \ln x + x^2 - 8 = 0$. Iterate until the absolute error $< 10^{-6}$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

- a. Find initial estimates



From the plot of $f(x) = 0$, we see that the root lies between 2 and 3. We set $l_0 = 2$ and $r_0 = 3$.

- b. EASY procedures

```

procedure MAIN()
  1: input  $L, R$  //initial interval//
  call HALF_INTERVAL( $10^{-6}$ , 20,  $L, R$ )
  go to 1
end procedure

procedure  $f(x)$ 
  return  $2 * \ln(x) + x^2 - 8$ 
end procedure

```

c. Results

Given equation: $2 \ln x + x^2 - 8 = 0$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of half-interval method:

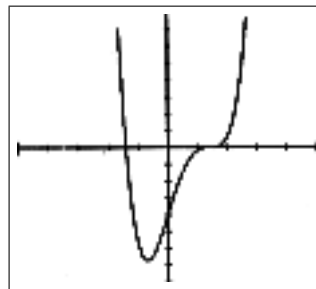
Initial interval: $[2, 3]$

k	l_k	r_k	x_k	$r_k - l_k$
0	2.0000000	3.0000000	2.5000000	1.0000000
1	2.0000000	2.5000000	2.2500000	.5000000
2	2.2500000	2.5000000	2.3750000	.2500000
3	2.3750000	2.5000000	2.4375000	.1250000
4	2.4375000	2.5000000	2.4687500	.0625000
5	2.4687500	2.5000000	2.4843750	.0312500
6	2.4843750	2.5000000	2.4921875	.0156250
7	2.4843750	2.4921875	2.4882813	.0078125
8	2.4843750	2.4882813	2.4863281	.0039063
9	2.4843750	2.4863281	2.4853516	.0019531
10	2.4853516	2.4863281	2.4858398	.0009766
11	2.4853516	2.4858398	2.4855957	.0004883
12	2.4855957	2.4858398	2.4857178	.0002441
13	2.4857178	2.4858398	2.4857788	.0001221
14	2.4857178	2.4857788	2.4857483	.0000610
15	2.4857178	2.4857483	2.4857330	.0000305
16	2.4857178	2.4857330	2.4857254	.0000153
17	2.4857254	2.4857330	2.4857292	.0000076
18	2.4857292	2.4857330	2.4857311	.0000038
19	2.4857311	2.4857330	2.4857321	.0000019
20	2.4857321	2.4857330	2.4857326	.0000010

Example 2. Find the roots of the equation $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$. Iterate until the absolute error $< 10^{-7}$ or the number of iterations exceeds 30, whichever occurs first.

Solution:

a. Find initial estimates



From the plot of $f(x) = 0$, we see that there is a distinct negative root between -2 and -1 and three equal positive roots between 1 and 2 .

b. EASY procedures

procedure MAIN()

1: input L, R

call HALF_INTERVAL(10^{-7} , 30, L, R)

go to 1

end procedure

//initial interval//

```

procedure f(x)
  return  $x^4 - 2 * \sqrt{2} * x^3 + 4 * \sqrt{2} * x - 4$ 
end procedure

```

c. Results (see also Newton's method Example 1, Richmond's method, Example 1)

Given equation: $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$

$itmax = 30, \varepsilon = 10^{-7}$

Iterations of half-interval method:

Initial interval: $[-2, -1]$

k	l_k	r_k	x_k	$r_k - l_k$
0	-2.0000000	-1.0000000	-1.5000000	1.0000000
1	-1.5000000	-1.0000000	-1.2500000	.5000000
2	-1.5000000	-1.2500000	-1.3750000	.2500000
3	-1.5000000	-1.3750000	-1.4375000	.1250000
4	-1.4375000	-1.3750000	-1.4062500	.0625000
$\vdots$				
22	-1.4142137	-1.4142134	-1.4142135	.0000002
23	-1.4142137	-1.4142135	-1.4142136	.0000001
24	-1.4142136	-1.4142135	-1.4142136	.0000001

Initial interval: $[1, 2]$

k	l_k	r_k	x_k	$r_k - l_k$
0	1.0000000	2.0000000	1.5000000	1.0000000
1	1.0000000	1.5000000	1.2500000	.5000000
2	1.2500000	1.5000000	1.3750000	.2500000
3	1.3750000	1.5000000	1.4375000	.1250000
4	1.3750000	1.4375000	1.4062500	.0625000
$\vdots$				
21	1.4142127	1.4142132	1.4142129	.0000005
22	1.4142129	1.4142132	1.4142131	.0000002
23	1.4142131	1.4142132	1.4142131	.0000001
24	1.4142131	1.4142131	1.4142131	.0000001

Example 3. Roots of Legendre polynomials.

Let $P_n(x)$ be the Legendre polynomial of degree n . Then

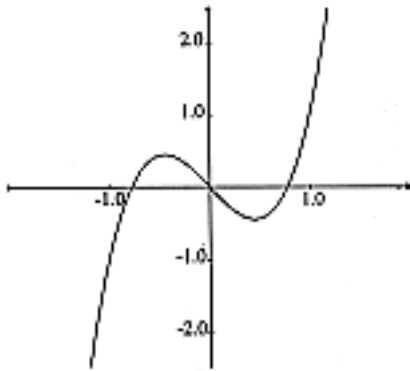
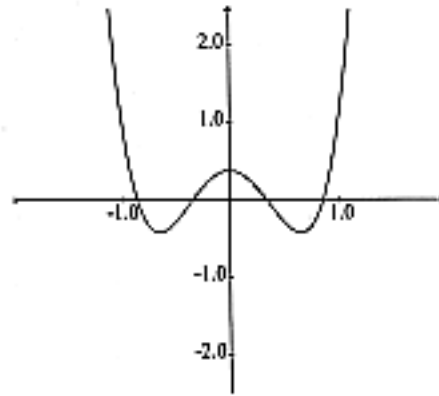
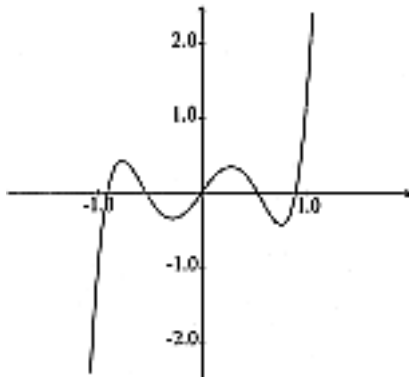
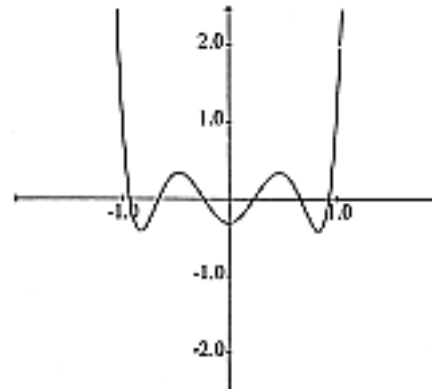
$$\begin{aligned}
 P_n(x) &= \sum_{k=0}^m \frac{(-1)^k (2n-2k)!}{2^k k! (n-k)! (n-2k)!} x^{(n-2k)}, \quad m = \left\lfloor \frac{n}{2} \right\rfloor \\
 &= \frac{2n-1}{n} \cdot x \cdot P_{n-1}(x) - \frac{n-1}{n} \cdot P_{n-2}(x)
 \end{aligned}$$

Examples

$$\begin{aligned}
 P_0(x) &= 1 \\
 P_1(x) &= x \\
 P_2(x) &= \frac{1}{2}(3x^2 - 1) \\
 P_3(x) &= \frac{1}{2}(5x^3 - 3x) \\
 P_4(x) &= \frac{1}{8}(35x^4 - 30x^2 + 3)
 \end{aligned}$$

$$P_5(x) = \frac{1}{8}(63x^5 - 70x^3 + 15x)$$

$$P_6(x) = \frac{1}{16}(231x^6 - 315x^4 + 105x^2 - 5)$$

$$\vdots$$

 $n = 3$

 $n = 4$

 $n = 5$

 $n = 6$

Some pertinent properties of Legendre polynomials

1. All n roots are real and lie in the interval $[-1, 1]$.
2. The roots are symmetrically arranged about the origin. This means that if n is odd, then zero is a root.

The roots of Legendre polynomials are used as base points in the *Gauss-Legendre quadrature formulas* for numerical integration.

The half-interval method is a most suitable method for finding all the roots of a Legendre polynomial since they all lie within a well-defined interval.

Solution:

- a. Only $\lfloor n/2 \rfloor$ roots of $P_n(x)$ need to be calculated, say those in $[-1, 0]$. We choose an interval size δ and 'march' to the right in steps of δ until we find an interval which contains a root. We then apply the half-interval method to reduce δ to ε . The value of δ should not be too small (to avoid a 'long march') nor too large (to avoid having more than one root in the interval). A reasonable value is a fraction of the average spacing between roots.

- b. EASY procedures

procedure MAIN()

1: input $n, \varepsilon, itmax$

// n is assumed to be global //

$\delta \leftarrow \lfloor 0.5/(n+1) \rfloor$

$L \leftarrow -1$

for $j \leftarrow 1$ to $\lfloor n/2 \rfloor$ do

while $f(L) * f(L + \delta) > 0$ do

$L \leftarrow L + \delta$

end while

$R \leftarrow L + \delta$

call HALF_INTERVAL($\varepsilon, itmax, L, R$)

$L \leftarrow R$

end for

go to 1

end procedure

procedure f(x)

case

: $n = 2$: return $(3 * x^2 - 1)/2$

: $n = 3$: return $(5 * x^3 - 3 * x)/2$

: $n = 4$: return $(35 * x^4 - 30 * x^2 + 3)/8$

: $n = 5$: return $(63 * x^5 - 70 * x^3 + 15 * x)/8$

: $n = 6$: return $(231 * x^6 - 315 * x^4 + 105 * x^2 - 5)/16$

: else : [output ' n is out of range'; stop]

end case

end procedure

- c. Results

Iterations of the half-interval method for the Legendre polynomial of degree 6 ($\varepsilon = 1.0D - 15$)

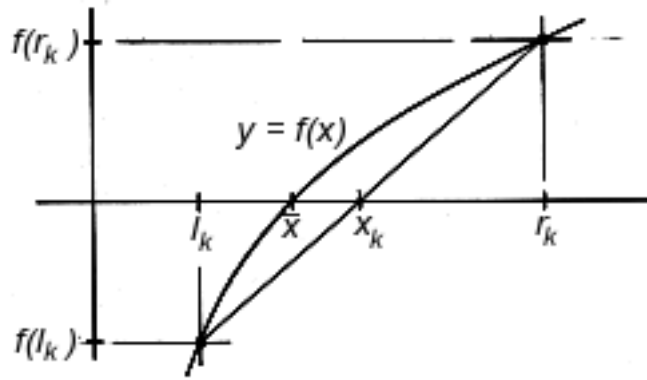
k	l_k	r_k	x_k	$r_k - l_k$
0	-1.0000000000000000	-.928571425378323	-.964285712689161	.714285746216774D - 01
1	-.964285712689161	-.928571425378323	-.946428569033742	.357142873108387D - 01
2	-.946428569033742	-.928571425378323	-.937499997206032	.178571436554194D - 01
3	-.937499997206032	-.928571425378323	-.933035711292177	.892857182770968D - 02
⋮				
44	-.932469514203153	-.932469514203149	-.932469514203151	.410782519111308D - 14
45	-.932469514203153	-.932469514203151	-.932469514203152	.199840144432528D - 14
46	-.932469514203152	-.932469514203151	-.932469514203152	.999200722162641D - 15
k	l_k	r_k	x_k	$r_k - l_k$
0	-.718183790338119	-.646755215716442	-.682469503027281	.714285746216774D - 01
1	-.682469503027281	-.646755215716442	-.664612359371861	.357142873108387D - 01
2	-.664612359371861	-.646755215716442	-.655683787544152	.178571436554194D - 01
3	-.664612359371861	-.655683787544152	-.660148073458006	.892857182770968D - 02
⋮				
44	-.661209386466265	-.661209386466261	-.661209386466263	.399680288865056D - 14
45	-.661209386466265	-.661209386466263	-.661209386466264	.199840144432528D - 14
46	-.661209386466265	-.661209386466264	-.661209386466265	.999200722162641D - 15

k	l_k	r_k	x_k	$r_k - l_k$
0	-.304066513357877	-.232637938736200	-.268352226047039	.714285746216774D - 01
1	-.268352226047039	-.232637938736200	-.250495082391619	.357142873108387D - 01
2	-.250495082391619	-.232637938736200	-.241566510563910	.178571436554194D - 01
3	-.241566510563910	-.232637938736200	-.237102224650055	.892857182770968D - 02
⋮				
45	-.238619186083198	-.238619186083196	-.238619186083197	.202615701994091D - 14
46	-.238619186083197	-.238619186083196	-.238619186083197	.102695629777827D - 14
47	-.238619186083197	-.238619186083197	-.238619186083197	.499600361081320D - 14

1.10 Regula-falsi method

Regula-falsi is also known as method of false position, and is also called linear interpolation method.

1.10.1 Derivation



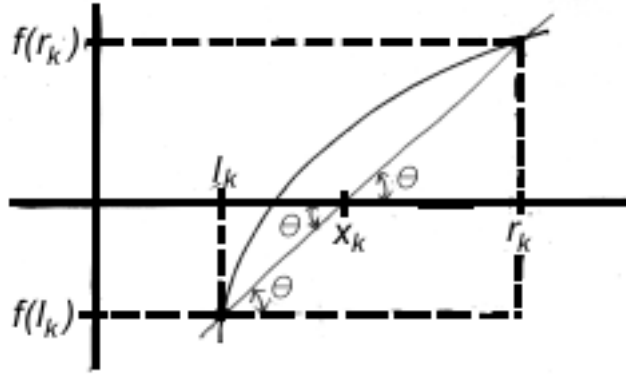
By similar triangles:

$$\frac{r_k - x_k}{f(r_k)} = \frac{r_k - l_k}{f(r_k) - f(l_k)}$$

Solving for x_k , we obtain:

$$\begin{aligned}
 x_k &= \frac{l_k \cdot f(r_k) - r_k \cdot f(l_k)}{f(r_k) - f(l_k)} \Leftarrow \text{traditional form (standard)} \\
 &= \frac{l_k \cdot f(r_k)}{f(r_k) - f(l_k)} - \frac{r_k \cdot f(l_k)}{f(r_k) - f(l_k)} + l_k - l_k \cdot \frac{f(r_k) - f(l_k)}{f(r_k) - f(l_k)} \\
 &= l_k + \frac{l_k \cdot f(r_k)}{f(r_k) - f(l_k)} - \frac{r_k \cdot f(l_k)}{f(r_k) - f(l_k)} - \frac{l_k[f(r_k) - f(l_k)]}{f(r_k) - f(l_k)} \\
 &= l_k + \frac{l_k \cdot f(r_k)}{f(r_k) - f(l_k)} - \frac{r_k \cdot f(l_k)}{f(r_k) - f(l_k)} - \frac{l_k \cdot f(r_k)}{f(r_k) - f(l_k)} + \frac{l_k \cdot f(l_k)}{f(r_k) - f(l_k)} \\
 &= l_k + \frac{f(l_k) \cdot (l_k - r_k)}{f(r_k) - f(l_k)} \Leftarrow \text{alternative form (requires one less multiplication than the standard form)}
 \end{aligned}$$

1.10.2 Alternative Derivation of the Alternate Form



$$\begin{aligned}\tan \theta &= \frac{f(l_k)}{l_k - x_k} = \frac{f(r_k) - f(l_k)}{r_k - l_k} \\ \frac{l_k - x_k}{f(l_k)} &= \frac{r_k - l_k}{f(r_k) - f(l_k)}\end{aligned}$$

Solving for x_k , we get:

$$x_k = l_k - \frac{f(l_k) \cdot (r_k - l_k)}{f(r_k) - f(l_k)} = l_k + \frac{f(l_k) \cdot (l_k - r_k)}{f(r_k) - f(l_k)}$$

1.10.3 Algorithm

1. Choose l_k and r_k , $k = 0$, such that $f(l_k)f(r_k) < 0$.

2. Calculate

$$x_k = \frac{l_k \cdot f(r_k) - r_k \cdot f(l_k)}{f(r_k) - f(l_k)}$$

3. If $f(l_k)f(x_k) > 0$, set $l_{k+1} = x_k$ and $r_{k+1} = r_k$; else, set $l_{k+1} = l_k$ and $r_{k+1} = x_k$

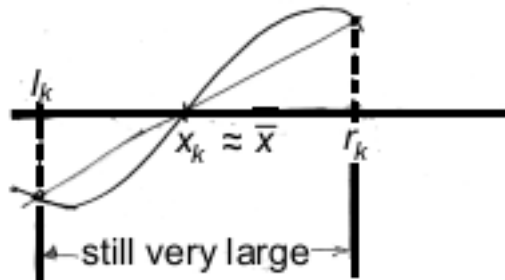
4. Repeat steps 2 and 3 until

a. Relative error criterion: $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| < \varepsilon$ or

b. $|f(x_k)| < \varepsilon$ or

c. the number of iterations $> itmax$

The absolute error criterion $r_k - l_k$ is not suitable for the *regula-falsi* method. The estimate may be already be very close to $\bar{x}$ but $r_k - l_k$ may still be very large. The test $|f(x)| < \varepsilon$ appears to be the most appropriate.



1.10.4 EASY Implementation

```

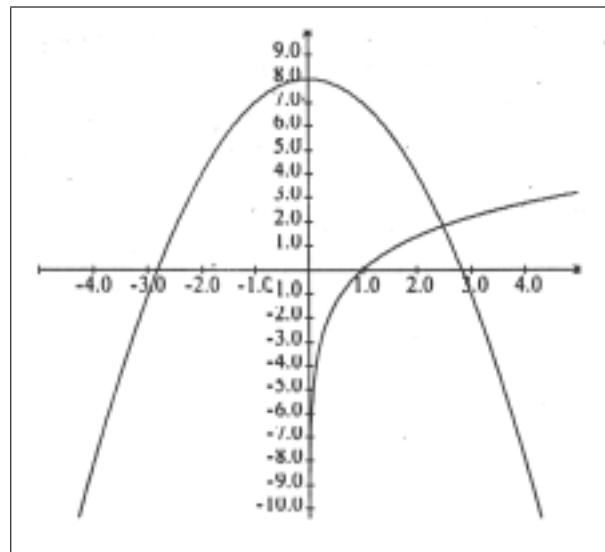
procedure REGULA_FALSI( $\varepsilon$ ,  $itmax$ ,  $L$ ,  $R$ )
   $fL \leftarrow f(L)$ 
   $fR \leftarrow f(R)$ 
  for  $iter \leftarrow 0$  to  $itmax$  do
    //Computation//
     $x \leftarrow (L * fR - R * fL) / (fR - fL)$ 
     $fx \leftarrow f(x)$ 
    output  $iter, L, R, x, fx$ 
    //Test for termination//
    if  $|fx| < \varepsilon$  then return
    //Updating//
    if  $fL * fx > 0$  then [ $L \leftarrow x$ ;  $fL \leftarrow fx$ ]
    else [ $R \leftarrow x$ ;  $fR \leftarrow fx$ ]
  end for
  return //  $\varepsilon$  not satisfied in  $itmax$  iterations //
end procedure

```

Example 1. Find the root of the equation $2 \ln x + x^2 - 8 = 0$. Iterate until $|f(x)| < 10^{-6}$ or the number of iterations exceeds 20, whichever occurs first.

Solution:

- a. Find initial estimates



We set $l_0 = 2$ and $r_0 = 3$.

```

b. EASY procedures
procedure MAIN()
  1: input  $L, R$  //initial interval//
  call REGULA_FALSI( $10^{-6}$ , 20,  $L, R$ )
  go to 1
end procedure

procedure  $f(x)$ 
  return  $2 * \ln(x) + x^2 - 8$ 
end procedure

```


c. Results

Given equation: $2 \ln x + x^2 - 8 = 0$

$itmax = 20, \varepsilon = 10^{-6}$

Iterations of *regula-falsi* method:

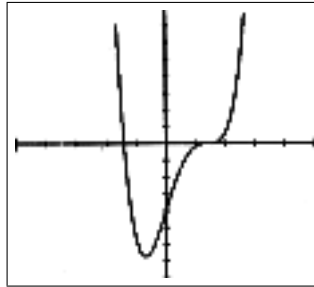
Initial interval: $[2, 3]$

k	l_k	r_k	x_k	$ f(x_k) $
0	2.0000000	3.0000000	2.4497913	0.2065171
1	2.4497913	3.0000000	2.4831744	0.0147695
2	2.4831744	3.0000000	2.4855509	0.0010482
3	2.4855509	3.0000000	2.4857195	0.0000744
4	2.4857195	3.0000000	2.4857314	0.0000053
5	2.4857314	3.0000000	2.4857323	0.0000004

Example 2. Find the roots of the equation $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$. Iterate until $|f(x)| < 10^{-7}$ or the number of iterations exceeds 30, whichever occurs first.

Solution:

a. Find initial estimates



From the plot of $f(x) = 0$, we see that there is a distinct negative root between -2 and -1 and three equal positive roots between 1 and 2 .

b. EASY procedures

procedure MAIN()

1: input L, R

call REGULA_FALSI(10^{-7} , 30, L, R)

go to 1

end procedure

procedure $f(x)$

return $x^4 - 2 * \sqrt{2} * x^3 + 4 * \sqrt{2} * x - 4$

end procedure

//initial interval//

c. Results (see also Newton's method Example 1, Richmond's method, Example 1)

Given equation: $x^4 - 2\sqrt{2}x^3 + 4\sqrt{2}x - 4 = 0$

$itmax = 30, \varepsilon = 10^{-7}$

Iterations of *regula-falsi* method:

Initial interval: $[-2, -1]$

k	l_k	r_k	x_k	$ f(x_k) $
0	-2.0000000	-1.0000000	-1.2000000	3.8271030
1	-2.0000000	-1.2000000	-1.3128073	2.0565063
2	-2.0000000	-1.3128073	-1.3685111	0.9848056
3	-2.0000000	-1.3685111	-1.3941050	0.4453696
4	-2.0000000	-1.3941050	-1.4054626	0.1961789
$\vdots$				
19	-2.0000000	-1.4142135	-1.4142135	0.0000007
20	-2.0000000	-1.4142135	-1.4142135	0.0000003
21	-2.0000000	-1.4142135	-1.4142136	0.0000001
22	-2.0000000	-1.4142136	-1.4142136	0.0000001

Initial interval: $[1, 2]$

k	l_k	r_k	x_k	$ f(x_k) $
0	1.0000000	2.0000000	1.2000000	0.0256970
1	1.2000000	2.0000000	1.2288735	0.0168275
2	1.2288735	2.0000000	1.2473286	0.0123704
3	1.2473286	2.0000000	1.2606553	0.0096855
4	1.2606553	2.0000000	1.2709443	0.0078964
5	1.2709443	2.0000000	1.2792373	0.0066234
$\vdots$				
497	1.3943998	2.0000000	1.3944191	0.0000218
498	1.3944191	2.0000000	1.3944383	0.0000217
499	1.3944383	2.0000000	1.3944575	0.0000217
500	1.3944575	2.0000000	1.3944766	0.0000216
$\vdots$				

Example 3.

Iterations of the *regula-falsi* method for the Legendre polynomial of degree 6 ($\varepsilon = 1.0E - 15$)

k	l_k	r_k	x_k	$ f(x_k) $
0	-1.0000000000000000	-.928571425378323	-.931043560959287	-.133497073864128D - 01
1	-1.0000000000000000	-.931043560959287	-.931951982076715	-.487673700001607D - 02
2	-1.0000000000000000	-.931951982076715	-.932282223862327	-.176902130462632D - 02
3	-1.0000000000000000	-.932282223862327	-.932401806504425	-.640069208769073D - 03
$\vdots$				
28	-1.0000000000000000	-.932469514203150	-.932469514203151	-.608974676241658D - 14
29	-1.0000000000000000	-.932469514203151	-.932469514203152	-.188998122707673D - 14
30	-1.0000000000000000	-.932469514203152	-.932469514203152	-.840473524110763D - 15

k	l_k	r_k	x_k	$ f(x_k) $
0	-.718183790338120	-.646755215716442	-.660657257594325	.173174649775357D - 02
1	-.718183790338120	-.660657257594325	-.661193188688707	.508363257247090D - 04
2	-.718183790338120	-.661193188688707	-.661208916885283	.147379557694644D - 05
3	-.718183790338120	-.661208916885283	-.661209372857669	.427110542237583D - 07
4	-.718183790338120	-.661209372857669	-.661209386071887	.123776642602970D - 08
5	-.718183790338120	-.661209386071887	-.661209386454836	.358703208195449D - 10
6	-.718183790338120	-.661209386454836	-.661209386465933	.103951959887216D - 11
7	-.718183790338120	-.661209386465933	-.661209386466255	.300677451686715D - 13
8	-.718183790338120	-.661209386466255	-.661209386466264	.797322277645840D - 15

k	l_k	r_k	x_k	$ f(x_k) $
0	-.304066513357868	-.232637938736191	-.238687901784039	.146293711950872D - 03
1	-.238687901784039	-.232637938736191	-.238619064960429	-.257862337553098D - 06
2	-.238687901784039	-.238619064960429	-.238619186081093	-.447854104604524D - 11
3	-.238687901784039	-.238619186081093	-.238619186083197	-.674102700742862D - 16

Roots of Legendre polynomials: summary

n	True value	Half-interval method	$Regula-falsi$ method
2	± 0.577350269189626	± 0.577350269189626	48 ± 0.577350269189626 12
3	0.0 ± 0.774596669241483	0.0 ± 0.774596669241483	47 ± 0.774596669241483 17
4	± 0.861136311594053 ± 0.339981043584856	± 0.861136311594053 ± 0.339981043584857	47 ± 0.861136311594053 15 47 ± 0.339981043584856 6
5	0.0 ± 0.906179845938664 ± 0.538469310105683	0.0 ± 0.906179845938664 ± 0.538469310105683	47 ± 0.906179845938664 11 47 ± 0.538469310105683 8
6	± 0.932469514203152 ± 0.661209386466265 ± 0.238619186083197	± 0.932469514203152 ± 0.661209386466265 ± 0.238619186083197	46 ± 0.932469514203152 30 46 ± 0.661209386466264 8 47 ± 0.238619186083197 3

Note: Initial interval size $\delta = 0.5/(n + 1)$

$\varepsilon = 1.0D - 15$

1.11 Graeffe's Root-Squaring Method

1.11.1 Form of the Equation

$$\begin{aligned}
 f(x) &= x^n + a_1x^{n-1} + a_2x^{n-2} + \cdots + a_{n-1}x + a_n = 0 \quad a_n \neq 0 \\
 &= (x - \bar{x}_1)(x - \bar{x}_2)(x - \bar{x}_3) \cdots (x - \bar{x}_n)
 \end{aligned}$$

1.11.2 Steps in Graeffe's Root-Squaring Method

1. Generating the k th derived polynomial $g(y)$ from the given polynomial $f(x)$ such that
 - (a) $f(x)$ and $g(y)$ have the same degree
 - (b) the roots of $g(y) = 0$ are the negatives of some large even power, m of the roots of $f(x) = 0$, that is

$$\begin{aligned}\bar{y}_1 &= -\bar{x}_1^m & m &= 2^k \\ \bar{y}_2 &= -\bar{x}_2^m \\ &\vdots \\ \bar{y}_n &= -\bar{x}_n^m\end{aligned}\tag{1.11}$$

2. Finding the roots of the k th derived polynomial $g(y) = 0$ from the simple *approximate* relationships between its coefficients and its roots.
3. Calculating the roots of $f(x) = 0$ from Eqs. (1.11).

Computationally steps 2 and 3 are carried out as a single step.

1.11.3 Root-Squaring Algorithm

Consider the polynomial equation

$$f(x) = 0 = x^2 + x - 6 = (x + 3)(x - 2)$$

with roots $\bar{x}_1 = -3$ and $\bar{x}_2 = +2$. Now, form the polynomial

$$f(x)f(-x) = 0 = (x^2 + x - 6)(x^2 - x - 6) = x^4 - 13x^2 + 36$$

Letting $p = -x^2$, we get

$$f(x)f(-x) = 0 = h(p) = p^2 + 13p + 36 = (p + 9)(p + 4)$$

We note that

$$\begin{aligned}\bar{p}_1 &= -9 = -(-3)^2 = -\bar{x}_1^2 \\ \bar{p}_2 &= -4 = -(+2)^2 = -\bar{x}_2^2\end{aligned}$$

Now form the polynomial equation

$$h(p)h(-p) = 0 = (p^2 + 13p + 36)(p^2 - 13p + 36) = p^4 - 97p^2 + 1296$$

Letting $q = -p^2$, we get

$$h(p)h(-p) = 0 = s(q) = q^2 + 97q + 1296 = (q + 81)(q + 16)$$

We note that

$$\begin{aligned}\bar{q}_1 &= -81 = -(-9)^2 = -(-3)^4 = -\bar{x}_1^4 \\ \bar{q}_2 &= -16 = -(-4)^2 = -(+2)^4 = -\bar{x}_2^4\end{aligned}$$

Repeating this root-squaring process on successive derived polynomials, we obtain

$$\begin{aligned}k = 3: & -\bar{x}_1^8 = -6561 \\ & -\bar{x}_2^8 = -256 \\ k = 4: & -\bar{x}_1^{16} = -43046721 \\ & -\bar{x}_2^{16} = -65536 \\ k = 5: & -\bar{x}_1^{32} = -1.853020 \times 10^{15} \\ & -\bar{x}_2^{32} = -4.294967 \times 10^9\end{aligned}$$

Inspection of the above results show that at some large even power m , say 64, we will have $\bar{x}_1^m \gg \bar{x}_2^m$. This is the key to Graeffe's root-squaring method.

In general for the n th degree polynomial equation

$$f(x) = \sum_{i=0}^n a_i^0 x^{n-i} = 0, \quad a_0^0 = 1 \quad (1.12)$$

the k th derived polynomial equation, which is obtained after k root-squaring steps, will be

$$\begin{aligned} g(y) &= \sum_{i=0}^n a_i^k y^{n-i} = 0, \quad a_0^k = 1 \\ &= \sum_{i=0}^n b_i y^{n-i} = 0, \quad b_0 = 1 \end{aligned} \quad (1.13)$$

where $b_i = a_i^k$, for every i . The roots of the two equations are related as found in Eqs. (1.11).

Let two successive polynomials be defined by the coefficients

$$\begin{array}{cccccccccccc} a_0^j & a_1^j & a_2^j & a_3^j & a_4^j & a_5^j & \cdots & a_{n-2}^j & a_{n-1}^j & a_n^j \\ a_0^{j+1} & a_1^{j+1} & a_2^{j+1} & a_3^{j+1} & a_4^{j+1} & a_5^{j+1} & \cdots & a_{n-2}^{j+1} & a_{n-1}^{j+1} & a_n^{j+1} \end{array}$$

Then:

$$\begin{aligned} a_0^{j+1} &= (a_0^j)^2 \\ a_1^{j+1} &= (a_1^j)^2 - 2a_0^j a_2^j \\ a_2^{j+1} &= (a_2^j)^2 - 2a_1^j a_3^j + 2a_0^j a_4^j \\ a_3^{j+1} &= (a_3^j)^2 - 2a_2^j a_4^j + 2a_1^j a_5^j - 2a_0^j a_6^j \\ &\vdots \\ a_{n-1}^{j+1} &= (a_{n-1}^j)^2 - 2a_{n-2}^j a_n^j \\ a_n^{j+1} &= (a_n^j)^2 \end{aligned} \quad (1.14)$$

To summarize: Starting with the original polynomial (1.12), we use (1.14) to generate k successive polynomials, culminating in (1.13). The roots of (1.12) and (1.13) are related according to (1.11).

1.11.4 Relations Between Coefficients and Roots of $g(y) = 0$

Consider the 3rd degree polynomial equation

$$f(x) = x^3 + a_1 x^2 + a_2 x + a_3 = 0$$

with roots $\bar{x}_1$, $\bar{x}_2$, and $\bar{x}_3$, such that $|\bar{x}_1| \geq |\bar{x}_2| \geq |\bar{x}_3|$. This can be expressed as

$$\begin{aligned} f(x) &= (x - \bar{x}_1)(x - \bar{x}_2)(x - \bar{x}_3) = 0 \\ &= x^3 - (\bar{x}_1 + \bar{x}_2 + \bar{x}_3)x^2 + (\bar{x}_1\bar{x}_2 + \bar{x}_1\bar{x}_3 + \bar{x}_2\bar{x}_3)x - \bar{x}_1\bar{x}_2\bar{x}_3 = 0 \end{aligned}$$

from which

$$\begin{aligned} a_1 &= -(\bar{x}_1 + \bar{x}_2 + \bar{x}_3) \\ a_2 &= \bar{x}_1\bar{x}_2 + \bar{x}_1\bar{x}_3 + \bar{x}_2\bar{x}_3 \\ a_3 &= -\bar{x}_1\bar{x}_2\bar{x}_3 \end{aligned}$$

Now define the Enke roots of a polynomial as the negatives of its roots, such that

$$\begin{aligned} r_1 &= -\bar{x}_1 \\ r_2 &= -\bar{x}_2 \\ &\vdots \\ r_n &= -\bar{x}_n \end{aligned}$$

In terms of the Enke roots, we therefore have

$$\begin{aligned} a_1 &= r_1 + r_2 + r_3 \\ a_2 &= r_1 r_2 + r_1 r_3 + r_2 r_3 \\ a_3 &= r_1 r_2 r_3 \end{aligned}$$

Thus, for the k th derived polynomial $g(y) = y^3 + b_1 y^2 + b_2 y + b_3 = 0$, in which $\bar{y}_i = -\bar{x}_i^m = -r_i^m$, $m = 2^k$, we have

$$\begin{aligned} b_1 &= -(\bar{y}_1 + \bar{y}_2 + \bar{y}_3) = r_1^m + r_2^m + r_3^m \\ b_2 &= \bar{y}_1 \bar{y}_2 + \bar{y}_1 \bar{y}_3 + \bar{y}_2 \bar{y}_3 = r_1^m r_2^m + r_1^m r_3^m + r_2^m r_3^m \\ b_3 &= -\bar{y}_1 \bar{y}_2 \bar{y}_3 = r_1^m r_2^m r_3^m \end{aligned}$$

In general, for the n th degree, k th derived polynomial equation (1.12):

$$\begin{aligned} b_1 &= r_1^m + r_2^m + r_3^m + \cdots + r_n^m \\ b_2 &= r_1^m r_2^m + r_1^m r_3^m + \cdots + r_1^m r_n^m + r_2^m r_3^m + r_2^m r_4^m + \cdots + \\ &\quad r_2^m r_n^m + \cdots + r_{n-1}^m r_n^m \\ b_3 &= r_1^m r_2^m r_3^m + r_1^m r_2^m r_4^m + \cdots + r_1^m r_2^m r_n^m + r_1^m r_3^m r_4^m + \\ &\quad r_1^m r_3^m r_5^m + \cdots + r_1^m r_3^m r_n^m + \cdots + r_{n-2}^m r_{n-1}^m r_n^m \\ &\vdots \\ b_n &= r_1^m r_2^m r_3^m \cdots r_n^m \end{aligned} \tag{1.15}$$

Note that Eqs. (1.15) relate the Enke roots of the original polynomial to the coefficients of the k th derived polynomial. By making suitable approximations based on the fact that the roots of (1.12) are all well separated, we can solve for the Enke roots, r_i , $i = 1, 2, \dots, n$; and consequently, for the actual roots, $\bar{x}_i$, $i = 1, 2, \dots, n$. The working approximate relations between the Enke roots r_i^m and the coefficients b_i depend on the nature of the roots $\bar{x}_i$ of the original polynomial.

1.11.5 Finding the Roots

CASE 1: Distinct, real roots

Consider again the 3rd degree polynomial equation

$$f(x) = x^3 + a_1 x^2 + a_2 x + a_3 = 0$$

with roots $\bar{x}_1$, $\bar{x}_2$, and $\bar{x}_3$ and Enke roots r_1 , r_2 and r_3 . Let the k th derived polynomial equation be

$$g(y) = y^3 + b_1 y^2 + b_2 y + b_3 = 0$$

Then, from Eqs. (1.15) we have

$$\begin{aligned} b_1 &= r_1^m + r_2^m + r_3^m \\ b_2 &= r_1^m r_2^m + r_1^m r_3^m + r_2^m r_3^m \\ b_3 &= r_1^m r_2^m r_3^m \end{aligned}$$

Now assume that $|\bar{x}_1| > |\bar{x}_2| > |\bar{x}_3|$. Then the above relations become

$$\begin{aligned} b_1 &\approx r_1^m \\ b_2 &\approx r_1^m r_2^m \end{aligned}$$

since $r_1^m \gg r_2^m \gg r_3^m$. Using these working approximate relations, we obtain the following expressions for the Enke roots r_i , for every i ,

$$\begin{aligned} r_1 &\approx \sqrt[m]{b_1} \\ r_2 &\approx \sqrt[m]{\frac{b_2}{r_1^m}} \approx \sqrt[m]{\frac{b_2}{b_1}} \\ r_3 &= \sqrt[m]{\frac{b_3}{r_1^m r_2^m}} \approx \sqrt[m]{\frac{b_3}{b_2}} \end{aligned}$$

In general, for an n th degree polynomial equation with distinct real roots, the Enke roots will be, approximately

$$r_i = \sqrt[m]{\frac{b_i}{b_{i-1}}} \quad i = 1, 2, \dots, n \quad (1.16)$$

where $b_0 = 1$. Eqs. (1.16) yield the magnitude of the actual roots $\bar{x}_i, i = 1, 2, \dots, n$. Substitution of $+r_i$ and $-r_i$ into Eq. (1.12) gives the sign of $\bar{x}_i$, for every i .

Example. Find all the roots of $f(x) = x^4 - 2x^3 - 13x^2 + 14x + 24 = 0$ using GRSM.

(a) Generate Graeffe's table

k	m	a_0^k	a_1^k	a_2^k	a_3^k	a_4^k
0	1	1.0	-2.00000D+00	-1.30000D+01	1.40000D+01	2.40000D+01
1	2	1.0	3.00000D+01	2.73000D+02	8.20000D+02	5.76000D+02
2	4	1.0	3.54000D+02	2.64810D+04	3.57904D+05	3.31776D+05
3	8	1.0	7.23540D+04	4.48511D+08	1.10524D+11	1.10075D+11
4	16	1.0	4.33808D+09	1.85169D+17	1.21168D+22	1.21166D+22
5	32	1.0	1.84486D+19	3.41823D+34	1.46811D+44	1.46811D+44
6	64	1.0	3.40282D+38	1.16842D+69	2.15536D+88	2.15536D+88
7	128	1.0	1.15792D+77	1.36521D+138	4.64557D+176	4.64557D+176

$$\begin{aligned} (a_1^6)^2 &= (3.40282D + 38)^2 = 1.15792D + 77 = a_1^7 \\ (a_2^6)^2 &= (1.16842D + 69)^2 = 1.36521D + 138 = a_2^7 \\ (a_3^6)^2 &= (2.15536D + 88)^2 = 4.64558D + 176 \approx a_3^7 \end{aligned} \Rightarrow \begin{array}{l} \text{Regular squared} \\ \text{relationship: all roots} \\ \text{real and distinct} \end{array}$$

(b) Find Enke roots from the equations $r_i = \sqrt[m]{\frac{b_i}{b_{i-1}}}, i = 1, 2, \dots, n$ for real, distinct roots.

$$\begin{aligned} r_1 &= \sqrt[m]{\frac{b_1}{b_0}} = \sqrt[128]{1.15792D + 77} = 4.00000 \\ r_2 &= \sqrt[m]{\frac{b_2}{b_1}} = \sqrt[128]{\frac{1.36521D + 138}{1.15792D + 77}} = 3.00000 \\ r_3 &= \sqrt[m]{\frac{b_3}{b_2}} = \sqrt[128]{\frac{4.64557D + 176}{1.36521D + 138}} = 2.00000 \\ r_4 &= \sqrt[m]{\frac{b_4}{b_3}} = \sqrt[128]{\frac{4.64557D + 176}{4.64557D + 176}} = 1.00000 \end{aligned}$$

(c) Find actual roots

r_i	$f(+r_i)$	$f(-r_i)$	$\bar{x}_i$
4.00000	0.0	144.0	+4.00000
3.00000	-24.0	0.0	-3.00000
2.00000	0.0	-24.0	+2.00000
1.00000	+24.0	0.0	-1.00000

CASE 2: Repeated real roots

Consider again the 3rd degree polynomial equation

$$f(x) = x^3 + a_1x^2 + a_2x + a_3 = 0$$

with roots $\bar{x}_1, \bar{x}_2$, and $\bar{x}_3$ and Enke roots r_1, r_2 and r_3 . Let the k th derived polynomial equation be

$$g(y) = y^3 + b_1y^2 + b_2y + b_3 = 0$$

Then, we have

$$\begin{aligned} b_1 &= r_1^m + r_2^m + r_3^m \\ b_2 &= r_1^m r_2^m + r_1^m r_3^m + r_2^m r_3^m \\ b_3 &= r_1^m r_2^m r_3^m \end{aligned}$$

Now assume that $|\bar{x}_1| > |\bar{x}_2| = |\bar{x}_3|$. Then $r_2^m = r_3^m$ and for large m , $r_1^m \gg r_2^m$. The above relations can therefore be expressed approximately as

$$\begin{aligned} b_1 &= r_1^m + 2r_2^m \approx r_1^m \\ b_2 &= 2r_1^m r_2^m + r_2^{2m} \approx 2r_1^m r_2^m \\ b_3 &= r_1^m r_2^{2m} \end{aligned}$$

Performing the root-squaring step one more for the equation $g(y) = 0$, we obtain, say

$$h(z) = z^3 + c_1 z^2 + c_2 z + c_3 = 0$$

where

$$\begin{aligned} c_1 &= b_1^2 - 2b_0 b_2 \approx r_1^{2m} - 4r_1^m r_2^{2m} \approx r_1^{2m} \\ c_2 &= b_2^2 - 2b_1 b_3 \approx 4r_1^{2m} r_2^{2m} - 2r_1^{2m} r_2^{2m} \approx 2r_1^{2m} r_2^{2m} \\ c_3 &= b_3^2 = r_1^{2m} r_2^{4m} \end{aligned}$$

Comparing the coefficients of the two successive derived polynomial equations $g(y) = 0$ and $h(z) = 0$, we note that:

$$\begin{aligned} c_1 &\approx b_1^2 \\ c_2 &\approx \frac{1}{2} b_2^2 \Leftarrow \text{half-squared relationship} \\ c_3 &= b_3^2 \end{aligned}$$

We see that c_2 takes on a half-squared relationship with respect to the preceding coefficient b_2 . This indicates that $|\bar{x}_2|$ has multiplicity 2 and that, specifically, $|\bar{x}_2| = |\bar{x}_3|$ (as originally assumed). In general, if c_i takes on the half-squared relationship with respect to b_i , then it follows that $|\bar{x}_i| = |\bar{x}_{i+1}|$.

Following the same development, we can show that if $c_i = \frac{1}{3} b_i^2$ and $c_{i+1} = \frac{1}{3} b_{i+1}^2$, then $|\bar{x}_i| = |\bar{x}_{i+1}| = |\bar{x}_{i+2}|$.

It should be noted that equality of the roots is in terms of magnitude only; the roots may vary in sign. When a root, say $\bar{x}_i$, has multiplicity ≥ 3 , and both $+\bar{x}_i$ and $-\bar{x}_i$ satisfy $f(x) = 0$, Descartes's rule of signs may be used to determine the number of positive and negative roots.

Pursuing the example above, we calculate the Enke roots from the resulting approximate relations to yield

$$\begin{aligned} r_1 &= \sqrt[m]{b_1} \\ r_2 &= \sqrt[m]{\frac{b_2}{2r_1^m}} = \sqrt[m]{\frac{b_2}{2b_1}} = r_3 \end{aligned}$$

The actual roots are finally determined by substitution into $f(x) = 0$.

Example. Find all the roots of $f(x) = x^4 - 4x^3 - x^2 + 16x - 12 = 0$ using GRSM.

(a) Generate Graeffe's table

k	m	a_0^k	a_1^k	a_2^k	a_3^k	a_4^k
0	1	1.0	-4.00000D+00	-1.00000D+00	1.60000D+01	-1.20000D+01
1	2	1.0	1.80000D+01	1.05000D+02	2.32000D+02	1.44000D+02
2	4	1.0	1.14000D+02	2.96100D+03	2.35840D+04	2.07360D+04
3	8	1.0	7.07400D+03	3.43184D+06	4.33406D+08	4.29982D+08
4	16	1.0	4.31778D+07	5.64656D+12	1.84890D+17	1.84884D+17
5	32	1.0	1.85303D+15	1.59173D+25	3.41822D+34	3.41822D+34
6	64	1.0	3.43368D+30	1.26681D+50	1.16842D+69	1.16842D+69
7	128	1.0	1.17902D+61	8.02398D+99	1.36521D+138	1.36521D+138
8	256	1.0	1.39008D+122	3.21922D+199	1.86380D+276	1.86380D+276

$$\begin{aligned} (a_1^7)^2 &= (1.17902D + 61)^2 = 1.39009D + 122 \approx a_1^8 \Rightarrow \bar{x}_1 \text{ is real and distinct} \\ \frac{1}{2}(a_2^7)^2 &= \frac{1}{2}(8.02398D + 99)^2 = 3.21921D + 199 \approx a_2^8 \Rightarrow |\bar{x}_2| = |\bar{x}_3| \\ &\quad \bar{x}_4 \text{ is real and distinct} \end{aligned}$$

(b) Find Enke roots: $r_1^m \gg (r_2^m = r_3^m) \gg r_4^m$

$$\begin{aligned} b_1 &= r_1^m + r_2^m + r_3^m + r_4^m \approx r_1^m \\ b_2 &= r_1^m r_2^m + r_1^m r_3^m + r_1^m r_4^m + r_2^m r_3^m + r_2^m r_4^m + r_3^m r_4^m \approx 2r_1^m r_2^m \\ b_3 &= r_1^m r_2^m r_3^m + r_1^m r_2^m r_4^m + r_1^m r_3^m r_4^m + r_2^m r_3^m r_4^m \approx r_1^m r_2^{2m} \\ b_4 &= r_1^m r_2^m r_3^m r_4^m = r_1^m r_2^{2m} r_4^m \end{aligned}$$

Hence:

$$\begin{aligned} r_1 &= \sqrt[m]{b_1} = \sqrt[256]{1.39008D + 122} = 3.00000 \\ r_2 &= \sqrt[m]{\frac{b_2}{2b_1}} = \sqrt[256]{\frac{3.21922D + 199}{2(1.39008D + 122)}} = 2.00000 = r_3 \\ r_4 &= \sqrt[m]{\frac{b_4}{b_3}} = \sqrt[256]{\frac{1.86380D + 276}{1.86380D + 276}} = 1.00000 \end{aligned}$$

(c) Find actual roots: substitution into $f(x) = 0$ yields $\bar{x}_1 = 3.00000$, $\bar{x}_2 = 2.00000$, $\bar{x}_3 = -2.00000$ and $\bar{x}_4 = 1.00000$. Applying Descartes's rule of signs confirms that there are three positive roots and one negative root.

CASE 3: Complex roots

Consider now the polynomial equation

$$f(x) = x^4 + a_1x^3 + a_2x^2 + a_3x + a_4 = 0$$

with real roots $|\bar{x}_1| > |\bar{x}_2|$ and complex roots $\bar{x}_3$ and $\bar{x}_4$. Then we can write the last two roots as

$$\begin{aligned} \bar{x}_3 &= a + ib = Re^{i\theta} = R(\cos \theta + i \sin \theta) \\ \bar{x}_4 &= a - ib = Re^{-i\theta} = R(\cos \theta - i \sin \theta) \end{aligned}$$

where $R = \sqrt{a^2 + b^2}$, and $|\bar{x}_2| > |R|$. For the final derived polynomial $g(y) = 0$, we again have

$$\begin{aligned} b_1 &= r_1^m + r_2^m + r_3^m + r_4^m \\ b_2 &= r_1^m r_2^m + r_1^m r_3^m + r_1^m r_4^m + r_2^m r_3^m + r_2^m r_4^m + r_3^m r_4^m \\ b_3 &= r_1^m r_2^m r_3^m + r_1^m r_2^m r_4^m + r_1^m r_3^m r_4^m + r_2^m r_3^m r_4^m \\ b_4 &= r_1^m r_2^m r_3^m r_4^m \end{aligned}$$

Substituting the expressions for $\bar{x}_3$ and $\bar{x}_4$, and noting that $\cos m\theta = \frac{1}{2}(e^{im\theta} + e^{-im\theta})$, we obtain

$$\begin{aligned} b_1 &= r_1^m + r_2^m + 2R^m \cos m\theta \approx r_1^m \\ b_2 &= (r_1 r_2)^m + 2R^m(r_1^m + r_2^m) \cos m\theta + R^{2m} \approx (r_1 r_2)^m \\ b_3 &= 2(r_1 r_2 R)^m \cos m\theta + R^{2m}(r_1^m + r_2^m) \approx 2(r_1 r_2 R)^m \cos m\theta \\ b_4 &= (r_1 r_2 R^2)^m \end{aligned}$$

The approximations derive from the fact that $r_1^m \gg r_2^m \gg R^m$. From these relations we see that the sign of successive coefficients of a_j^j , $j = 1, 2, \dots, k$ will fluctuate because of the cosine term. This indicates that $\bar{x}_3$ is a complex root; its conjugate pair is $\bar{x}_4$. In general, if the sign of a_j^j , $j = 1, 2, \dots, k$ fluctuates, then $\bar{x}_i = a + ib$ and $\bar{x}_{i+1} = a - ib$ for some real a and b .

Example. Find all the roots of $f(x) = x^4 - 2x^3 - 14x^2 + 32x - 32 = 0$ using GRSM.

(a) Generate Graeffe's table

k	m	a_0^k	a_1^k	a_2^k	a_3^k	a_4^k
0	1	1.0	-2.00000D+00	-1.40000D+01	3.20000D+01	-3.20000D+01
1	2	1.0	3.20000D+01	2.60000D+02	1.28000D+02	1.02400D+03
2	4	1.0	5.04000D+02	6.14560D+04	-5.16096D+05	1.04858D+06
3	8	1.0	1.31104D+05	4.29916D+09	1.37473D+11	1.09951D+12
4	16	1.0	8.58994D+09	1.84467D+19	9.44473D+21	1.20893D+24
5	32	1.0	3.68935D+19	3.40282D+38	4.46015D+43	1.46150D+48
6	64	1.0	6.80565D+38	1.15792D+77	9.94646D+86	2.13599D+96
7	128	1.0	2.31584D+77	1.34078D+154	4.94661D+173	4.56255D+192

$$\frac{1}{2}(a_1^6)^2 = \frac{1}{2}(6.80565D + 38)^2 = 2.31584D + 77 = a_1^7 \Rightarrow |\bar{x}_1| = |\bar{x}_2|$$

$$a_3^j, j = 1, 2, \dots, 7 \text{ changes sign at } j = 2 \Rightarrow \bar{x}_3 = a + ib \text{ and } \bar{x}_4 = a - ib$$

(b) Find Enke roots: $(r_1^m = r_2^m) \gg R^m$ where $R = \sqrt{a^2 + b^2}$

$$\begin{aligned} b_1 &= r_1^m + r_2^m + r_3^m + r_4^m \approx 2r_1^m \\ b_2 &= r_1^m r_2^m + r_1^m r_3^m + r_1^m r_4^m + r_2^m r_3^m + r_2^m r_4^m + r_3^m r_4^m \approx r_1^{2m} \\ b_3 &= r_1^m r_2^m r_3^m + r_1^m r_2^m r_4^m + r_1^m r_3^m r_4^m + r_2^m r_3^m r_4^m \\ &\approx r_1^{2m} (Re^{i\theta})^m + r_1^{2m} (Re^{-i\theta})^m \approx r_1^{2m} R^m (e^{im\theta} + e^{-im\theta}) = 2r_1^{2m} R^m \cos m\theta \\ b_4 &= r_1^m r_2^m r_3^m r_4^m = r_1^{2m} R^{2m} \end{aligned}$$

Hence:

$$\begin{aligned} r_1 &= \sqrt[m]{\frac{b_1}{2}} = \sqrt[128]{\frac{2.31584D + 77}{2}} = 4.00000 = r_2 \\ R &= \sqrt[2m]{\frac{b_4}{b_2}} = \sqrt[256]{\frac{4.56244D + 192}{1.34078D + 154}} = 1.41421 = \sqrt{2} \end{aligned}$$

(c) Find actual roots: substitution into $f(x) = 0$ yields the real roots $\bar{x}_1 = 4.00000$ and $\bar{x}_2 = -4.00000$. To find the complex roots, we use the relation $a_1 = -(\bar{x}_1 + \bar{x}_2 + \bar{x}_3 + \bar{x}_4)$, or $-2 = -(4 - 4 + a + ib + a - ib)$, from which we obtain $a = 1.00000$. Finally, substitution into $R = \sqrt{a^2 + b^2}$ or $\sqrt{2} = \sqrt{(1.00000)^2 + b^2}$ yields $b = \pm 1.00000$. Hence, $\bar{x}_3 = 1.00000 + 1.00000i$ and $\bar{x}_4 = 1.00000 - 1.00000i$.

1.11.6 Horner's Rule

$$\begin{aligned} p_n(x) &= a_0x^n + a_1x^{n-1} + a_2x^{n-2} + \dots + a_{n-1}x + a_n \\ &= a_n + a_{n-1}x + a_{n-2}x^2 + \dots + a_2x^{n-2} + a_1x^{n-1} + a_0x^n \\ &= a_n + x(a_{n-1} + x(a_{n-2} + \dots + x(a_2 + x(a_1 + xa_0)))) \end{aligned}$$

Example

$$\begin{aligned} p_4(x) &= a_0x^4 + a_1x^3 + a_2x^2 + a_3x + a_4 \\ &= a_4 + a_3x + a_2x^2 + a_1x^3 + a_0x^4 \\ &= a_4 + x(a_3 + x(a_2 + x(a_1 + xa_0))) \end{aligned}$$

1.11.7 EASY Implementation

```

procedure GRAEFFE( $A, n, \varepsilon$ )
  array  $A(0:n), B(0:n), X(1:n), T(0:n)$ 
   $k \leftarrow 0$ 
   $m \leftarrow 1$ 
  //Normalize original polynomial//
   $temp \leftarrow A(0)$ 
  for  $i \leftarrow 0$  to  $n$  do
     $B(i) \leftarrow A(i)/temp$ 
  end for
  output  $k, m, B(0 : n)$ 
  //Perform root-squaring step//
   $switch \leftarrow 1$ 
  while  $switch > 0$  do
     $k \leftarrow k + 1$ 
    for  $i \leftarrow 1$  to  $n$  do
       $j \leftarrow i - 1$ 
       $l \leftarrow i + 1$ 
       $sign \leftarrow -1$ 
       $T(i) \leftarrow B(i)^2$ 
      while  $j \geq 0$  and  $l \leq n$  do
         $T(i) \leftarrow T(i) + sign * 2 * B(j) * B(l)$ 
         $j \leftarrow j - 1$ 
         $l \leftarrow l + 1$ 
         $sign \leftarrow -sign$ 
      end while
      if  $|T(i)| > 10^{150}$  or  $|T(i)| < 10^{-150}$  then  $switch \leftarrow 0$ 
    end for
     $m \leftarrow 2^k$ 
    for  $i \leftarrow 1$  to  $n$  do
       $B(i) \leftarrow T(i)$ 
    end for
    output  $k, m, B(0 : n)$ 
  end while
  //Calculate roots assuming that they are all real and distinct//
  for  $i \leftarrow 1$  to  $n$  do
     $X(i) \leftarrow |B(i)/B(i-1)|^{(1.0/m)}$ 
  end for
  //Determine sign and legitimacy of calculated roots//
  for  $i \leftarrow 1$  to  $n$  do
    1:  $fx \leftarrow f(A, x, X(i))$ 
    case
      :  $|fx| < \varepsilon$  : [output  $X(i), fx$ ; stop]
      :  $X(i) > 0$  : [ $X(i) \leftarrow -X(i)$ ; go to 1]
      : else : output  $X(i), fx, \text{'probably not a root'}$ 
    end case
  end for
end procedure

procedure f( $A, n, x$ )
  //Evaluates the polynomial  $f(x) = a_0x^n + a_1x^{n-1} + \dots + a_{n-1}x + a_n$  using Horner's rule//
  array  $A(0:n)$ 
   $f \leftarrow A(0)$ 
  for  $i \leftarrow 0$  to  $n$  do
     $f \leftarrow A(i) + x * f$ 
  end for
end procedure

```